# Computer System Design & Application

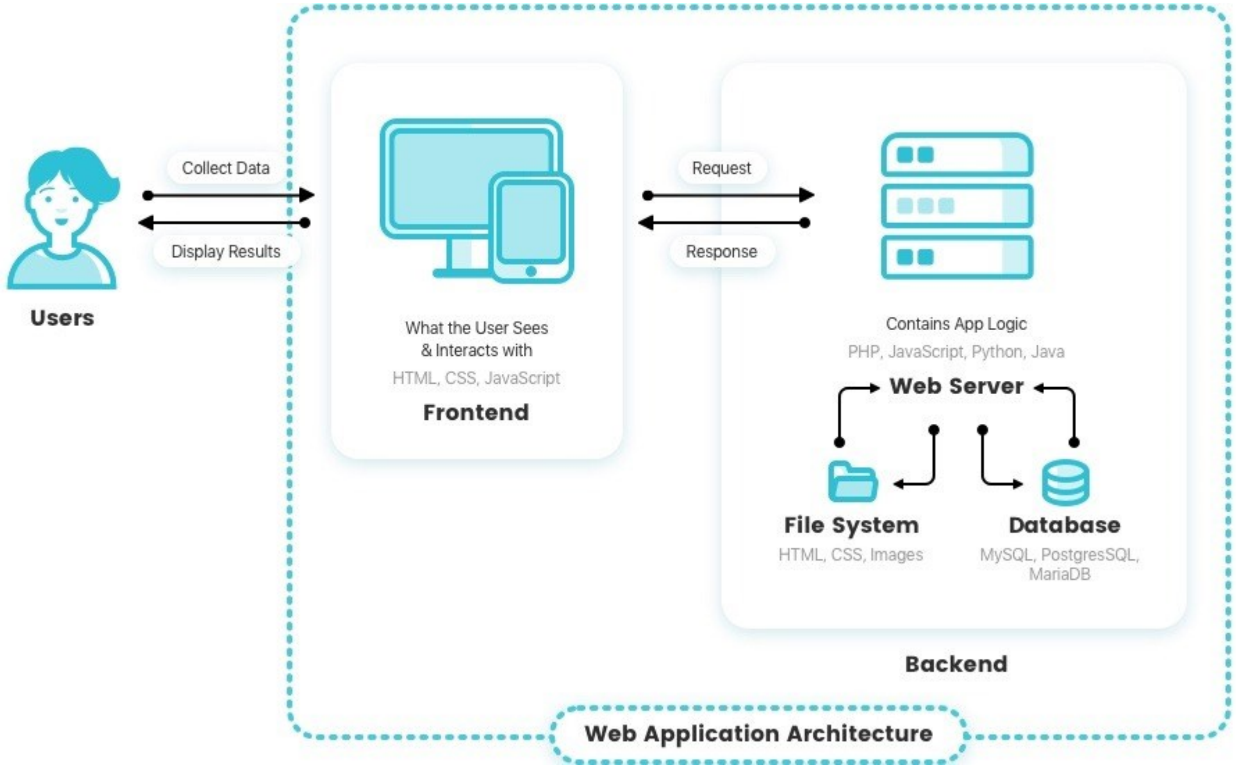
# 计算机系统设计与应用A

陶伊达 （TAO Yida）
taoyd@sustech.edu.cn

# Lecture 11

- Web Development Overview
- Java EE
- Servlet & Containers
- JDBC & JPA

# Web Application

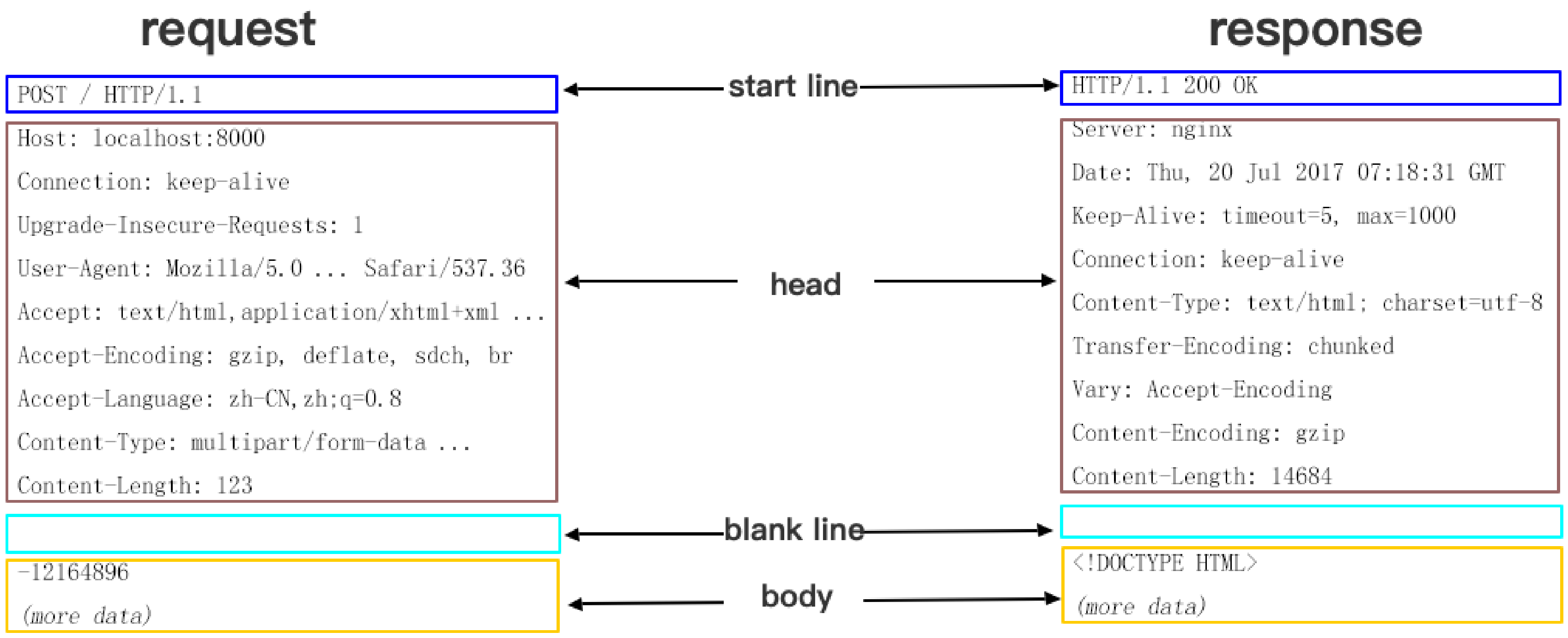



- A web application (or web app) is application software that runs on a web server, unlike computer-based software programs that are run locally on the OS of the device.
- Web applications are accessed by the user through a web browser with an active network connection. These applications are programmed using a client-server modeled structure
- Example web applications: web-mail, online retail sales, online banking, social network site, etc.

Image: https://reinventy.com/blog/fundamentals-web-application-architecture/

HTTP headers provide additional information about the data that will be sent





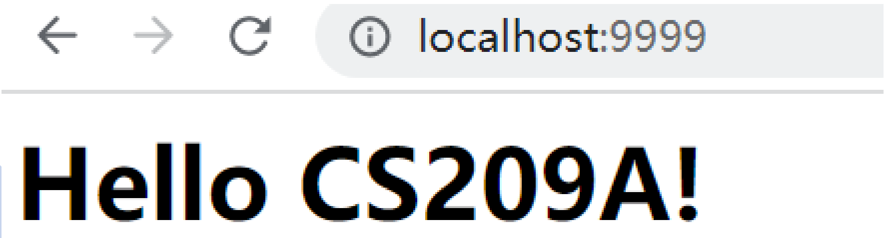



# Data Exchange on the Web

- HTTP is the set of rules (protocol) for transferring files (e.g., text, images, sound, video ) over the web
- Clients and servers exchange HTTP requests and responses, which follow specific syntax

# Building a Web Server with ServerSocket

- Reply a fixed html whenever client is connected to the server
- Need proper HTTP header information for clients to parse
- Type localhost:9999 in browser (or localhost if port is 80)




```java
public class ToyWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket server = new ServerSocket(9999);

        while (true) {
            Socket socket = server.accept();
            System.out.println("Client connected.");

            BufferedWriter writer = new BufferedWriter(
                    new OutputStreamWriter(socket.getOutputStream(),
                            StandardCharsets.UTF_8));

            String data = "<html><body><h2>Hello CS209A!</h2></body></html>";

            writer.write("HTTP/1.1 200 OK\r\n");
            writer.write("Content-Type: text/html; charset=UTF-8\r\n");
            writer.write("\r\n");
            writer.write(data);
            writer.flush();

        }
    }
}
```

# A web server is much more complex...

- We also have to:
  - Generate and parse correct HTTP headers/requests
  - Recognize and handle incorrect HTTP headers/requests
  - Handling concurrent requests
  - Handling network exceptions
  - Handling security issues
  - Handling performance issues
  - ....
- But we want to focus on application/business logic, instead of networking issues

# Division of Labor

**Application/Business Logic:**
**Developers**

**User Interface:**
**frontend developers/graphic designers**

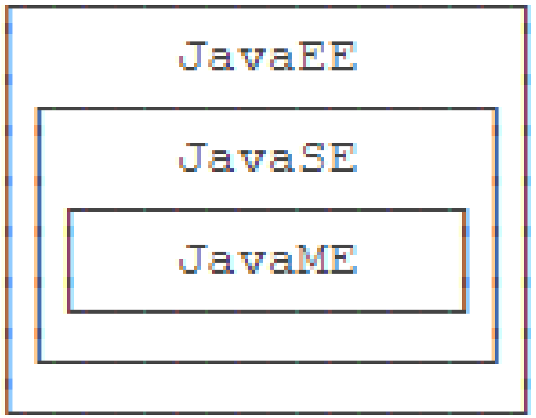
**Infrastructure:**
**Reusable web technologies/framework**

```
public class ToyWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket server = new ServerSocket(9999);

        while (true) {
            Socket socket = server.accept();
            System.out.println("Client connected.");

            BufferedWriter writer = new BufferedWriter(
                    new OutputStreamWriter(socket.getOutputStream(),
                            StandardCharsets.UTF_8));

            String data = "<html><body><h2>Hello CS209A!</h2></body></html>";

            writer.write("HTTP/1.1 200 OK\r\n");
            writer.write("Content-Type: text/html; charset=UTF-8\r\n");
            writer.write("\r\n");
            writer.write(data);
            writer.flush();

        }
    }
}
```

# Lecture 11

- Web Development Overview
- Java EE
- Servlet & Containers
- JDBC & JPA

# Java EE (Enterprise Edition)



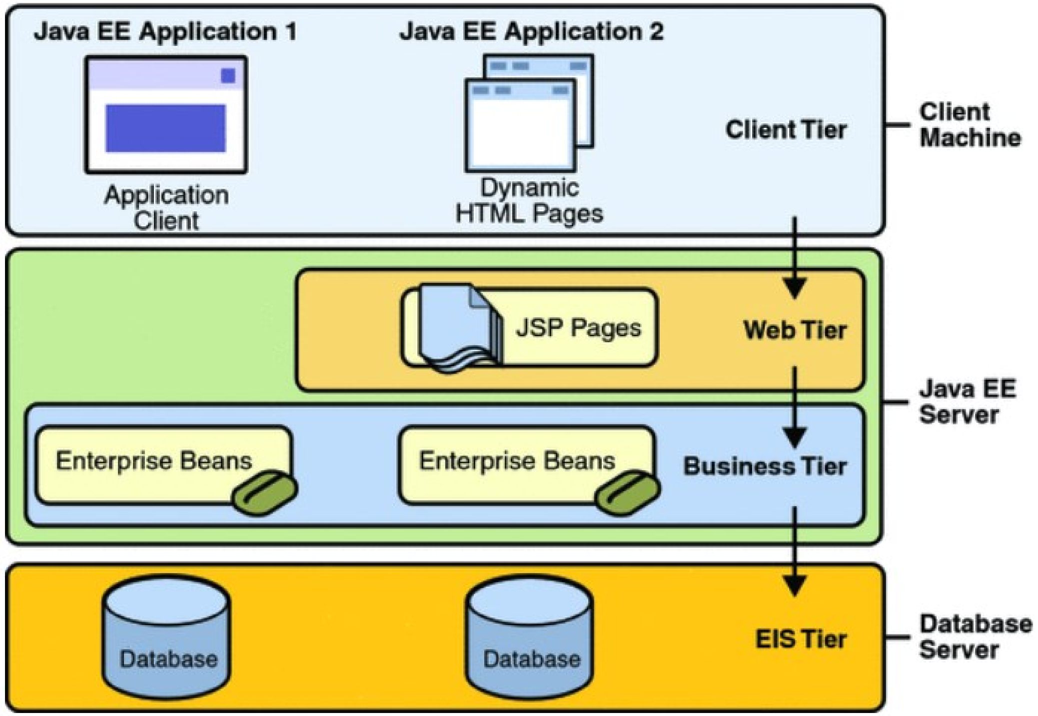

⋆ Formerly known as J2EE and now known as Jakarta EE

- The Java technologies you'll use to create web applications are a part of Java EE platform
- Java EE is built on top of Java SE (Standard Edition), which contains core APIs that we use daily (java.lang, java.io, etc.), and adds libraries for database access (JDBC, JPA), servlets, remote method invocation (RMI), messaging (JMS), web services, XML processing, Enterprise Beans, etc.
- Java EE provides APIs and runtime environment to help developers create large-scale, multi-tiered, scalable, reliable, and secure web/business applications

# Multitiered Applications

https://docs.oracle.com/javaee/7/firstcup/java-ee001.htm
https://docs.oracle.com/cd/E19575-01/819-3669/gfirp/index.html

- Java EE reduces the complexity of enterprise application by using a multitiered application model
- In a multi-tiered application, the functionality of the application is separated into isolated functional areas, called tiers; Typically, multi-tiered applications have a client tier, a middle tier, and a data tier




The client tier consists of a client program that makes requests to the middle tier

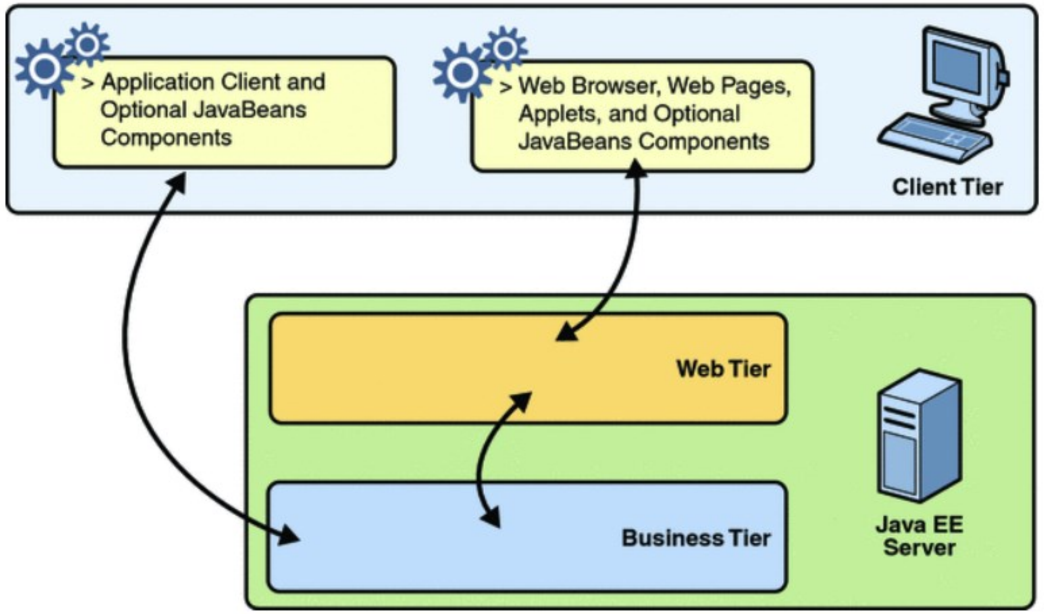
The middle tier is divided into a web tier and a business tier, which handle client requests and process application data, storing it in a permanent datastore in the data tier (often called the enterprise information systems tier).

# Client Tier (客户端层)

- A Java EE client can be a <u>web client</u> or an <u>application client</u>.

**Application client**

- runs on a client machine and typically has a GUI (e.g., created from Swing or AWT)
- Can directly access enterprise beans running in the business tier or communicate with a servlet running in the web tier
- Can be written in other languages and interact with Java EE servers



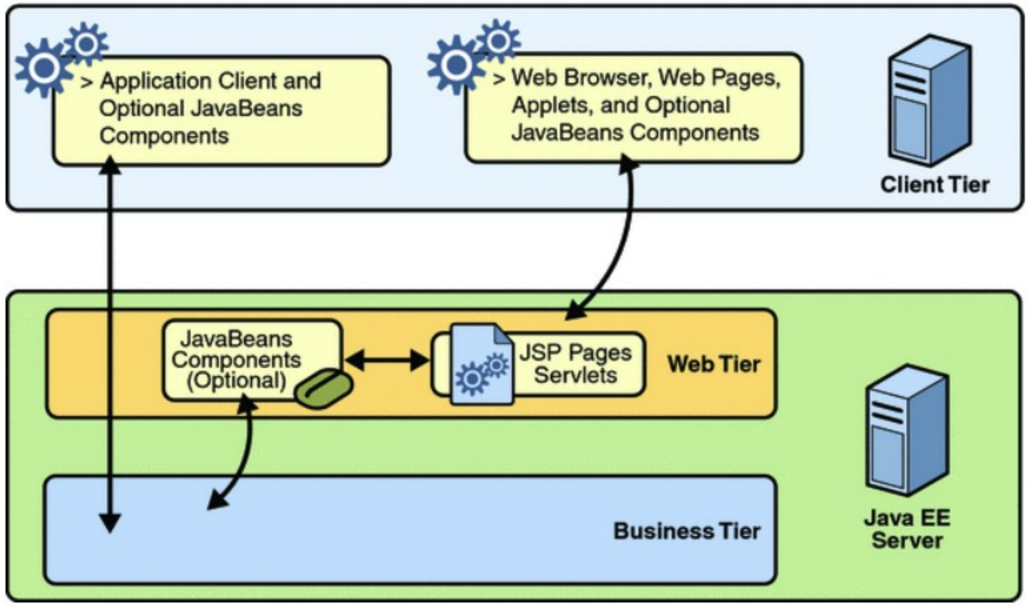

**Web client**

- consists of web pages and a web browser
- usually do not query databases, execute complex business rules
- A web page received from the web tier can include an embedded applet, a small client application written in Java that executes in JVM installed in the web browser

https://docs.oracle.com/cd/E19575-01/819-3669/gfirp/index.html

# Web Tier (Web层)

- The web tier consists of components that handle the interaction between clients and the business tier.



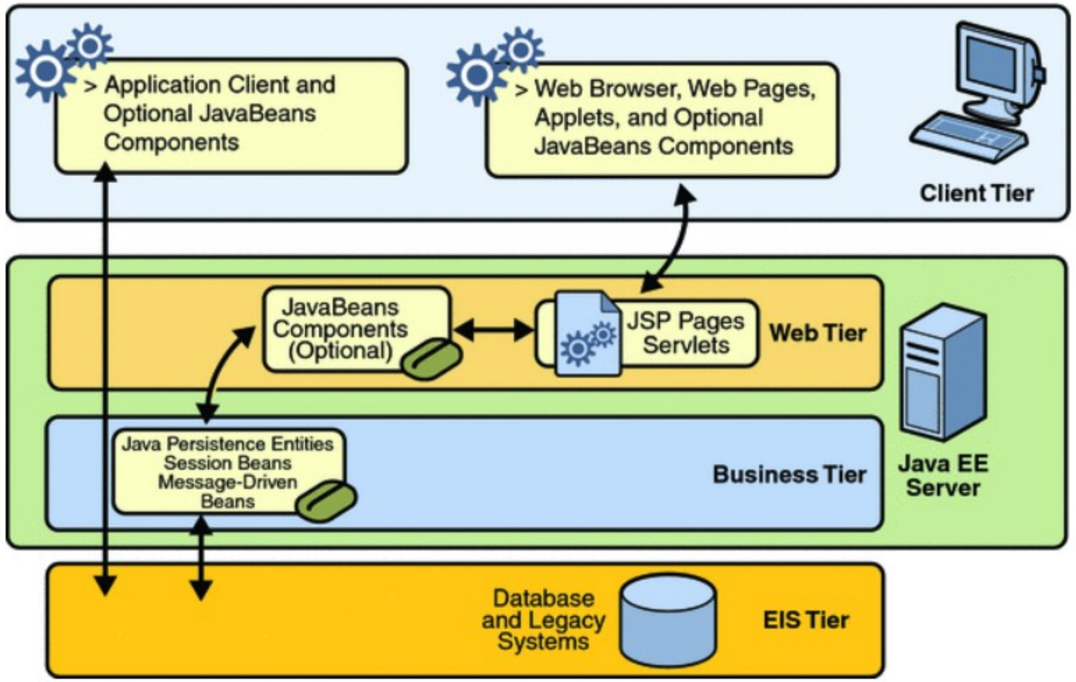

**Java EE web-tier technologies**

- **Servlet**: Java classes (APIs) that dynamically process requests and construct responses
- **JSP** (JavaServer Pages): extends/executes Servlet and intends to fullfill UI by generating web pages with HTML, XML, etc.

https://docs.oracle.com/cd/E19575-01/819-3669/gfirp/index.html

# Business Tier (业务层)

- Business code that solves or meets the needs of a particular business domain (e.g., banking, retail, or finance), is handled by enterprise beans running in the business tier
- In a properly designed enterprise application, the core functionality exists in the business tier components



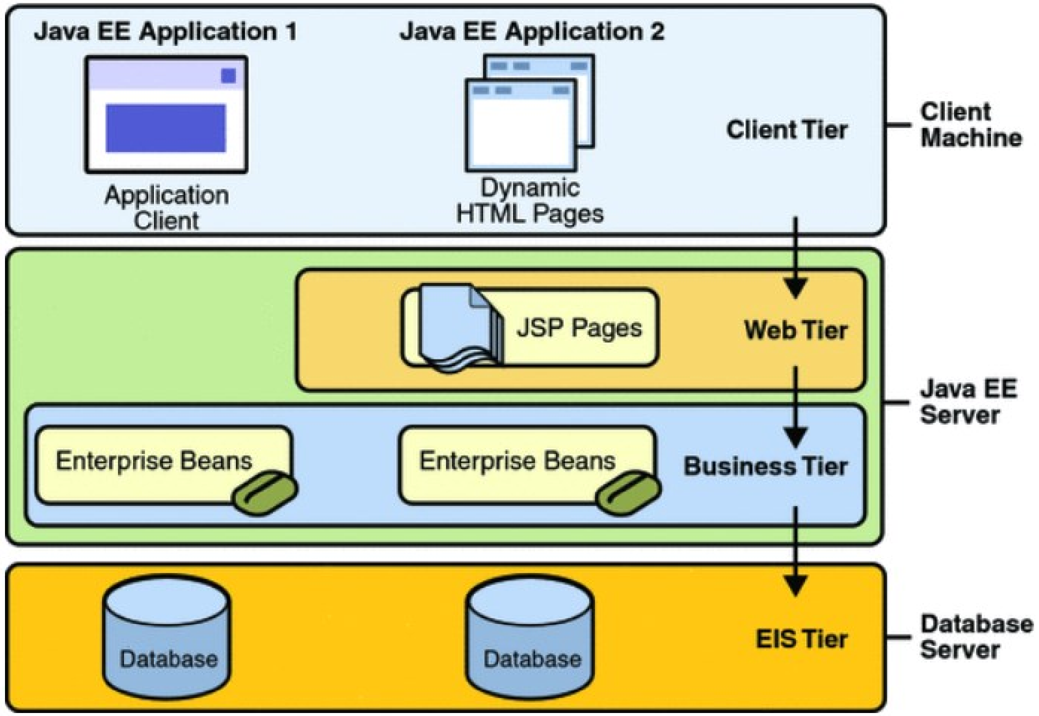

https://docs.oracle.com/cd/E19575-01/819-3669/gfirp/index.html

**Java EE Business-tier technologies**

**Enterprise JavaBean (EJB):** a server-side software component that provides many features and services such as transaction management, remote invocation, message-driven, security, lifecycle management, and load balancing, to facilitate the implementation of enterprise-level development.

# Data Tier (数据层)

- Also called the enterprise information systems (EIS) tier
- EIS consists of database servers, enterprise resource planning systems, and other legacy data sources, which typically locate on a separate machine from the Java EE server, and are accessed by the business tier




**Java EE data-tier technologies**

- The Java Database Connectivity API (JDBC)
- The Java Persistence API (JPA)
- The Java Transaction API (JTA)

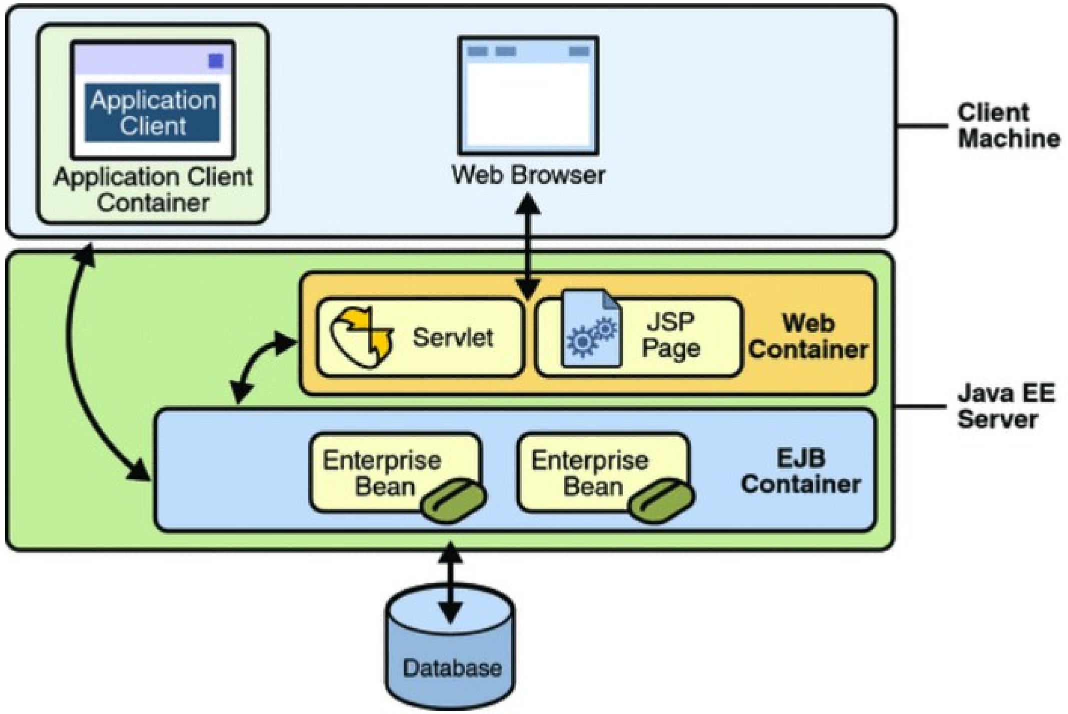
https://docs.oracle.com/javaee/7/firstcup/java-ee001.htm
https://docs.oracle.com/cd/E19575-01/819-3669/gfirp/index.html

# Java EE Servers, Components, Containers

- Java EE servers host several application component types (e.g., servlet, EJB) that correspond to the tiers in a multi-tiered application.
- Java EE server provides services to these components in the form of a **container**.
- Containers provide a standardized runtime environment with services such as concurrency management, lifecycle management, and request handling.

# Container Types

- Before a web component, enterprise bean, or application client component can be executed, it must be assembled into a Java EE module and deployed into its container




- **Enterprise JavaBeans (EJB) container**: Manages the execution of enterprise beans for Java EE applications. Enterprise beans and their container run on the Java EE server.
- **Web container**: Manages the execution of JSP page and servlet components for Java EE applications. Web components and their container run on the Java EE server.
- **Application client container**: Manages the execution of application client components. Application clients and their container run on the client.
- **Applet container**: Manages the execution of applets. Consists of a web browser and Java Plug-in running on the client together.

https://docs.oracle.com/cd/E19575-01/819-3669/gfirp/index.html

# Lecture 11

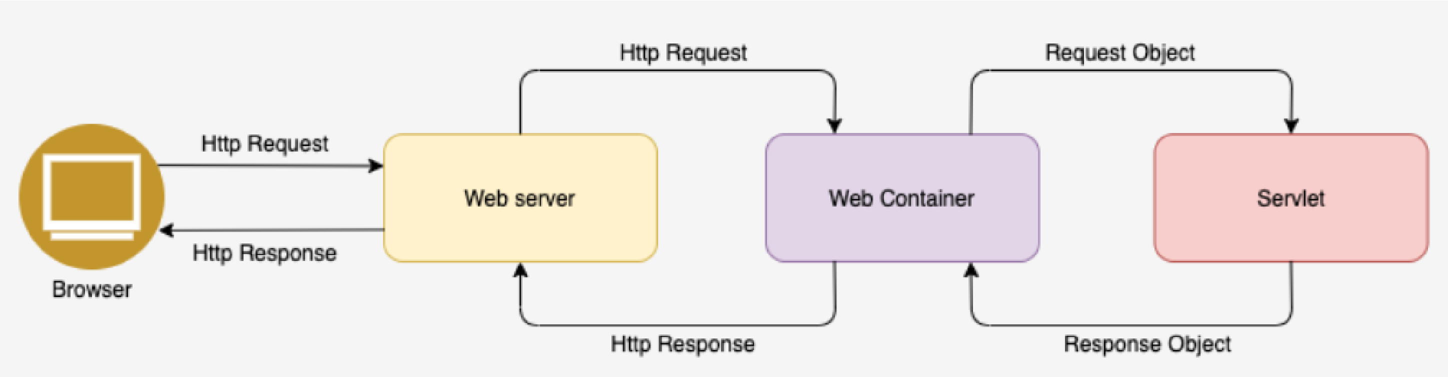
- Web Development Overview
- Java EE
- Servlet & Containers
- JDBC & JPA

# What is Servlet?

- Servlet is nothing but a Java program/class
- Servlets respond to incoming requests by implementing application or business logics
- Servlet can not understand raw requests; its a Java program, which only understands objects
- Servlets run in a servlet container, which provides a standardized runtime environment with services such as request handling, servlet lifecycle management, and concurrency.

# Workflow

- The client sends an HTTP Request to the web server
- Web server forwards requests to Web Container
- Web Container parse the HTTP request to objects and forward the request objects to the Servlet
- Servlet implement the application logic, builds the response object and sends it back to the Web Container
- Web container transforms the response object to equivalent HTTP response and sends it to the web server
- The web server sends the response via HTTP response back to the client.




https://codeburst.io/understanding-java-servlet-architecture-b74f5ea64bf4

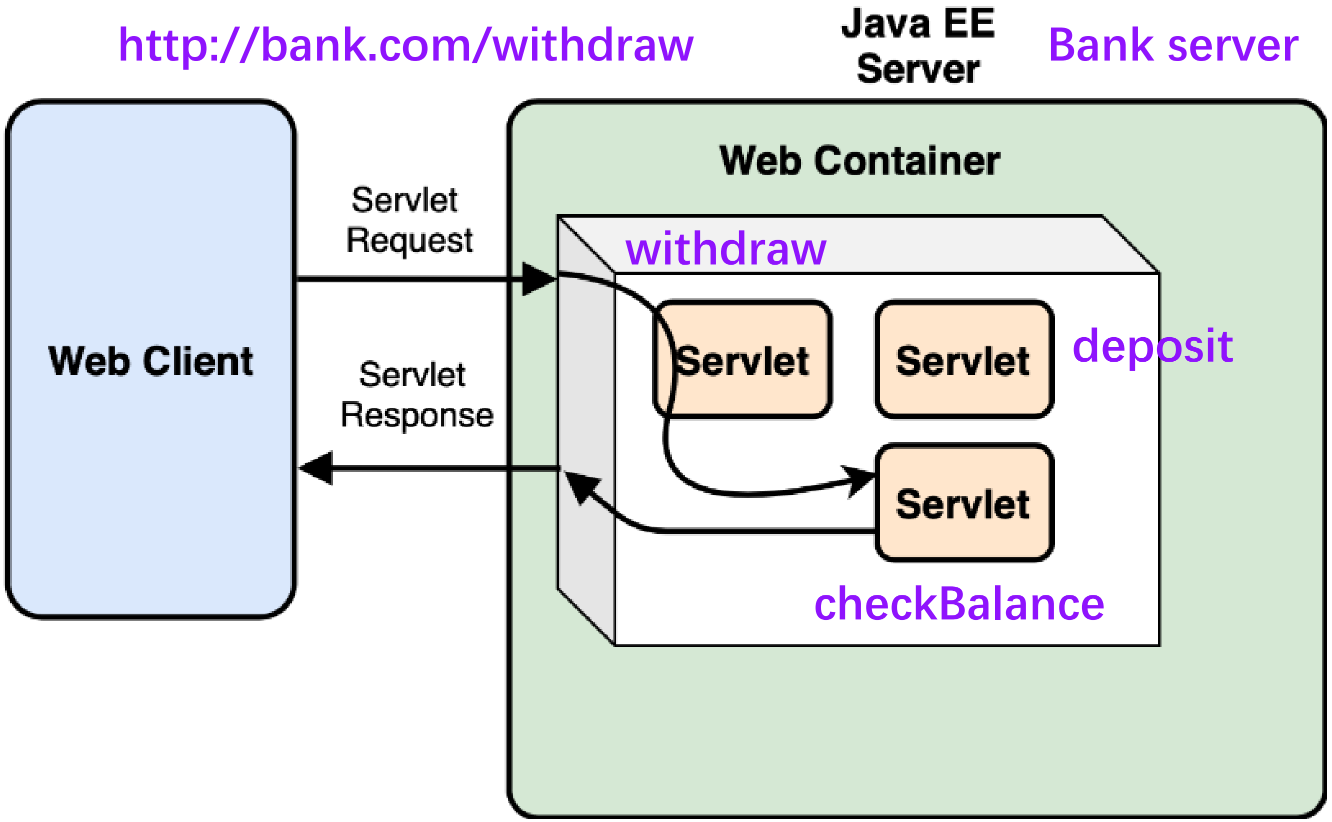
# Web Server vs. Web Container

- Web Server
  - Serves static files (e.g., HTML, images, text) via the HTTP protocol
  - You can write a very simple one in Java in a few lines of code; or using an open source one (e.g., Apache HTTPD)

- Web Container
  - Serves dynamic content by executing the server-side web component (e.g., servlet)
  - Convert HTTP requests to request objects and convert response objects to HTTP response

# How do containers & servlets work?

Mapping URL paths to corresponding servlets (typically by web.xml or annotations)




https://sergiomartinrubio.com/articles/get-started-with-java-servlets/

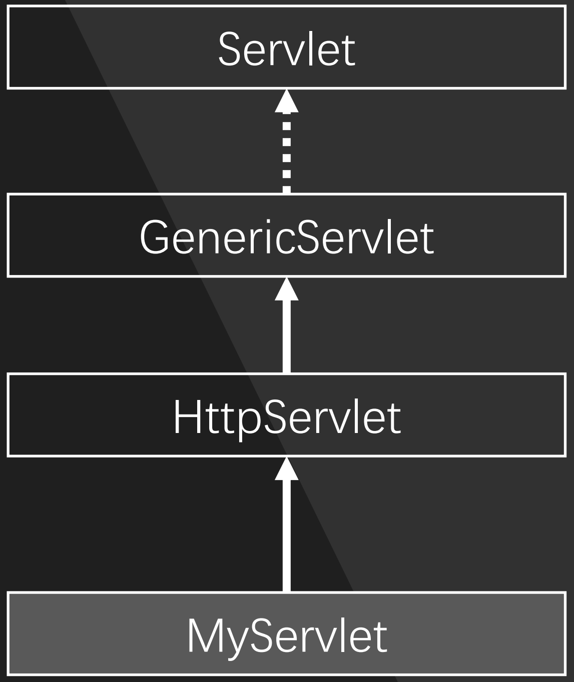
# The Servlet Interface

Defines methods that all servlets must implement

| Method and Description |
| --- |
| `destroy()`<br>Called by the servlet container to indicate to a servlet that the servlet is being taken out of service. |
| `getServletConfig()`<br>Returns a `ServletConfig` object, which contains initialization and startup parameters for this servlet. |
| `getServletInfo()`<br>Returns information about the servlet, such as author, version, and copyright. |
| `init(ServletConfig config)`<br>Called by the servlet container to indicate to a servlet that the servlet is being placed into service. |
| `service(ServletRequest req, ServletResponse res)`<br>Called by the servlet container to allow the servlet to respond to a request. |

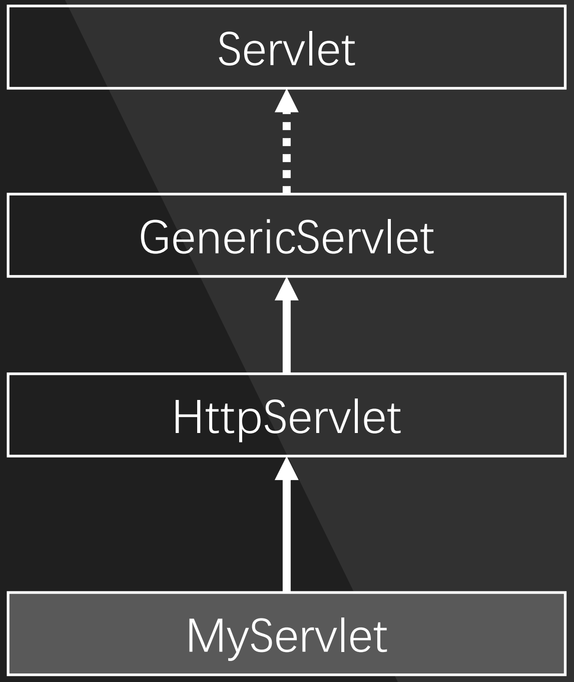
- Methods to initialize a servlet, to service requests, and to remove a servlet from the server
- Methods to get basic information and startup configuration
- The container invokes these methods

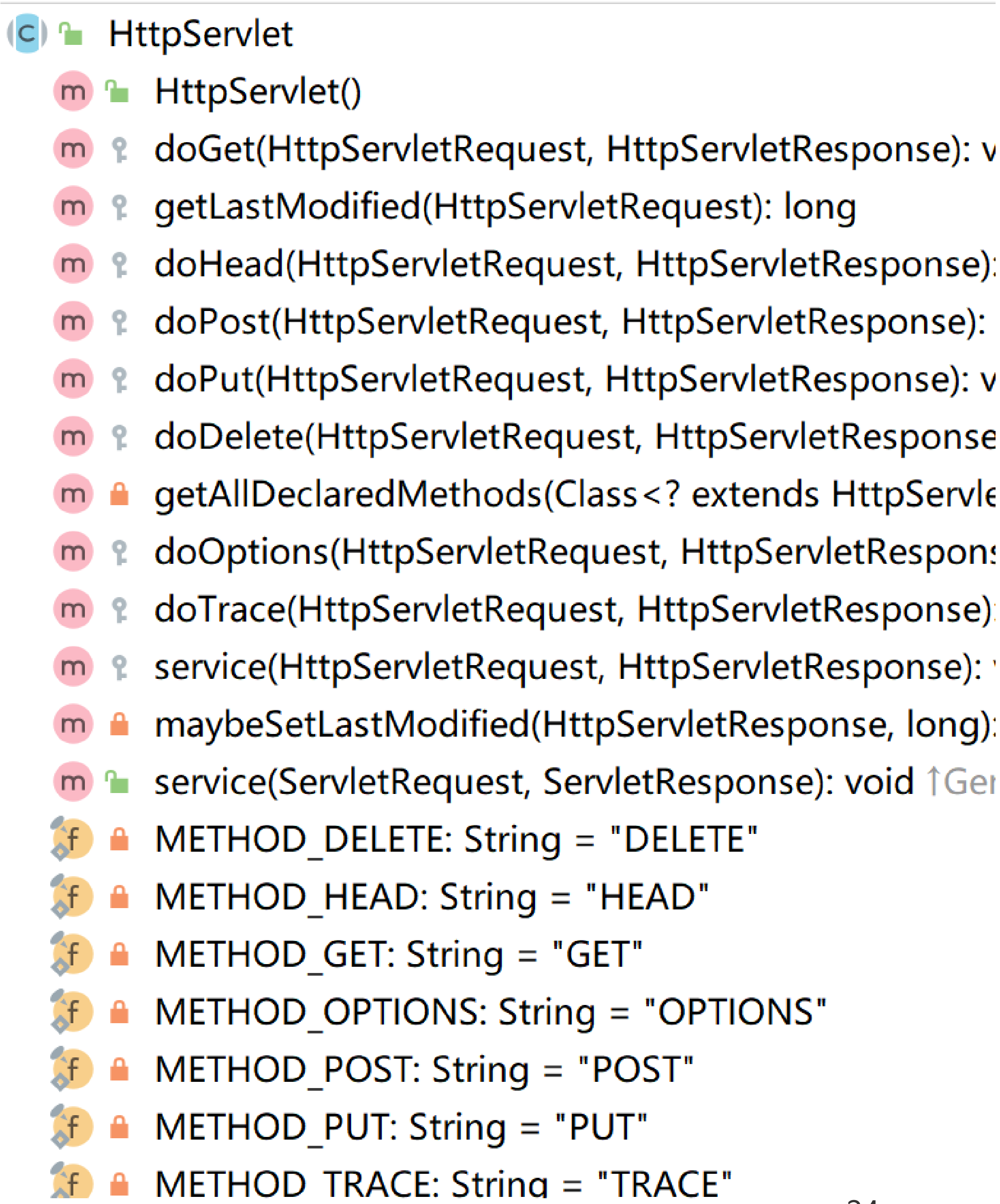
# The `HttpServlet` Class




- `GenericServlet` implements `Servlet`
  - An abstract class
  - A generic, protocol-independent servlet.
- `HttpServlet` extends `GenericServlet`
  - An abstract class
  - Defines a HTTP protocol specific servlet.
  - Adds fields and methods that are specific to HTTP protocol

# The `HttpServlet` Class




Typically, we would directly extend HttpServlet to create our own HTTP servlets

# The `HttpServlet` Class

- Provides an abstract class to be subclassed
- HttpServlet overrides service(), which dispatches the HTTP requests to corresponding methods (e.g., GET -> doGet())
- A subclass of HttpServlet must override at least one method, usually one of these:
    - `doGet`, if the servlet supports HTTP GET requests
    - `doPost`, for HTTP POST requests
    - `doPut`, for HTTP PUT requests
    - `doDelete`, for HTTP DELETE requests
    - `init` and `destroy`, to manage resources that are held for the life of the servlet

```java
protected void service(HttpServletRequest req, HttpServ
    String method = req.getMethod();
    long lastModified;
    if (method.equals("GET")) {...} else if (method.equ
        lastModified = this.getLastModified(req);
        this.maybeSetLastModified(resp, lastModified);
        this.doHead(req, resp);
    } else if (method.equals("POST")) {
        this.doPost(req, resp);
    } else if (method.equals("PUT")) {
        this.doPut(req, resp);
    } else if (method.equals("DELETE")) {
        this.doDelete(req, resp);
    } else if (method.equals("OPTIONS")) {
        this.doOptions(req, resp);
    } else if (method.equals("TRACE")) {
```

# The `HttpServlet` Class

- Provides an abstract class to be subclassed
- HttpServlet overrides service(), which dispatches the HTTP requests to corresponding methods (e.g., GET -> doGet())
- A subclass of HttpServlet must override at least one method, usually one of these:
    - `doGet`, if the servlet supports HTTP GET requests
    - `doPost`, for HTTP POST requests
    - `doPut`, for HTTP PUT requests
    - `doDelete`, for HTTP DELETE requests
    - `init` and `destroy`, to manage resources that are held for the life of the servlet

# Example

```java
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    2 usages
    private String message;

    public void init() { message = "Hello World!"; }

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        response.setContentType("text/html");

        // Hello
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>" + message + "</h1>");
        out.println("</body></html>");
    }

    public void destroy() {
    }
}
```

init() and destroy() manage resources that are held for the life of the servlet

**doGet**

- Called by the container (via the service method) to allow a servlet to handle a GET request.
- When overriding this method, read the request data, write the response headers, get the response's writer or output stream object, and finally, write the response data.
- It's best to include content type and encoding.

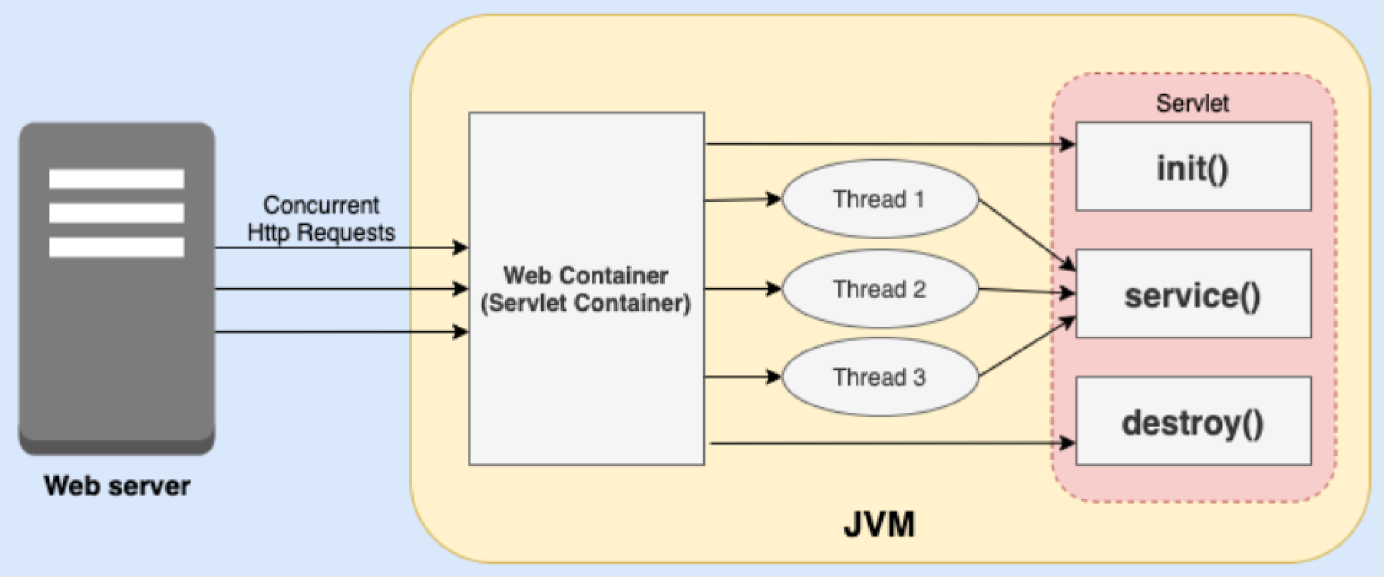
@WebServlet annotation is used to declare a servlet. The annotated class must extend the HttpServlet class.

- value: required, specify the URL of this servlet
- Name: optional, specify the name of this servlet

# Servlet Containers

- A servlet container is nothing but a compiled, executable program that runs on top of JVM
- The main function of the servlet container is to load, initialize, and execute servlets
- Servlet container manages the entire lifecycle of servlets

# Servlet Lifecycle




https://codeburst.io/understanding-java-servlet-architecture-b74f5ea64bf4

1. Concurrent HTTP requests coming to the server are forwarded to the web container.
2. The web container creates an instance of the servlet and executes init() (called only once)
3. The container handles multiple requests to the same instance by spawning multiple threads, each thread executing the service() method of a same instance of the servlet.
4. The container calls destroy() once all threads for a servlet exit; the servlet instance is removed from the container

# Multithreading

- A Java servlet container is typically multithreaded: multiple requests to the same servlet may be executed at the same time.
  - The container takes care of multithreading
- By default, a container may have only one instance per servlet declaration
  - The container handles concurrent requests to the same servlet by concurrent execution of the `service` method on different threads.
  - Application developers make sure that the servlet code is implemented to be thread-safe (i.e., accessing shared resource like instance/class variables )

# Where is `javax.servlet`?

Previously, we mostly use Java SE JDK

javax.servlet is part of Java EE; we should install Java EE SDK

Alternatively, simple servlet containers (e.g., Tomcat) also come with this API (servlet-api.jar).

# What exactly is Java EE?

# What exactly is Java EE?

- Java EE is indeed an abstract specification, which describes the standards, expected behaviors, and interactions between APIs (what we have learned so far)
  - APIs (e.g., the `Servlet` interface)
  - Natural-language specification (e.g., how to manage servlet lifecycle)
- Anybody (companies, providers, developers) is open to develop and provide a working, concrete implementation of (part of) the specification.
- An application is "Java EE compliant" if it meets the requirements of Java EE specification

# Reference Implementations

- In Java specifications' cases, you usually have a **reference implementation (RI)** created while drafting the specification

- Then other providers who may create their own implementation of the specification (often claiming it's "better" in some way).

- Java EE developers should write code following the specification (`import javax`…), then the code would run correctly on any concrete implementation.

# Java EE 6 RIs and Alternatives

- **Java EE full-fledged**
  - ***Oracle Glassfish (RI)***
  - **JBoss AS**
  - **IBM WebSphere**
- **Servlet & JSP**
  - *Oracle Glassfish (RI)*
  - Apache Tomcat
  - Eclipse Jetty
  - Resin
- **Enterprise JavaBeans (EJBs)**
  - *Oracle Glassfish (RI)*
  - Apache TomEE and OpenEJB
  - BuzyBeans
- **Java Persistence API (JPA)**
  - *EclipseLink (RI, used in Glassfish)*
  - OpenJPA
  - Hibernate

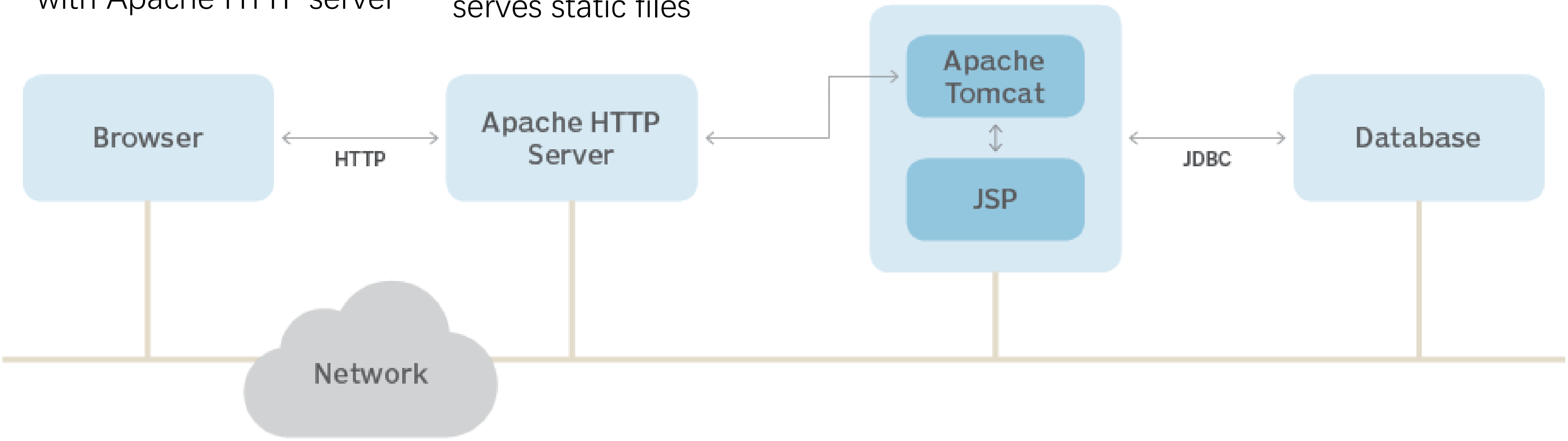
# Apache Tomcat

- Tomcat is a webcontainer which allows running servlet and JSP based web applications
- Tomcat is written in Java and requires JDK to run

Clients interact directly with Apache HTTP server

Apache HTTP server handles requests and serves static files

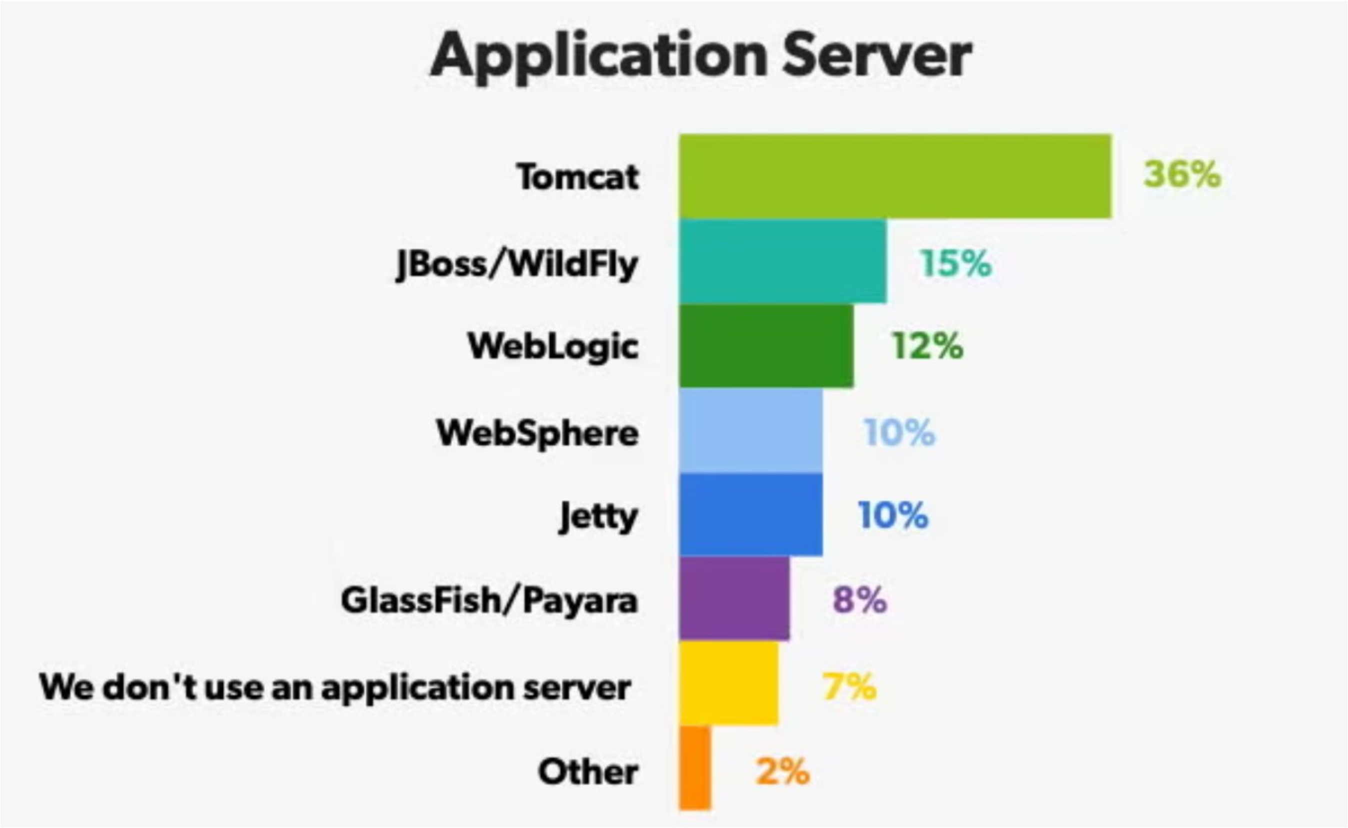
Requests that require logics are forwarded to Tomcat, which runs the servlets and interact with database




- Tomcat also has it's own HTTP server built into it, and is fully functional at serving static content too.
- But the performance of Tomcat as HTTP server is not as good as the performance of a designated web server, e.g., Apache HTTP server.
- For simple (production) applications, Tomcat alone is sufficient and good enough

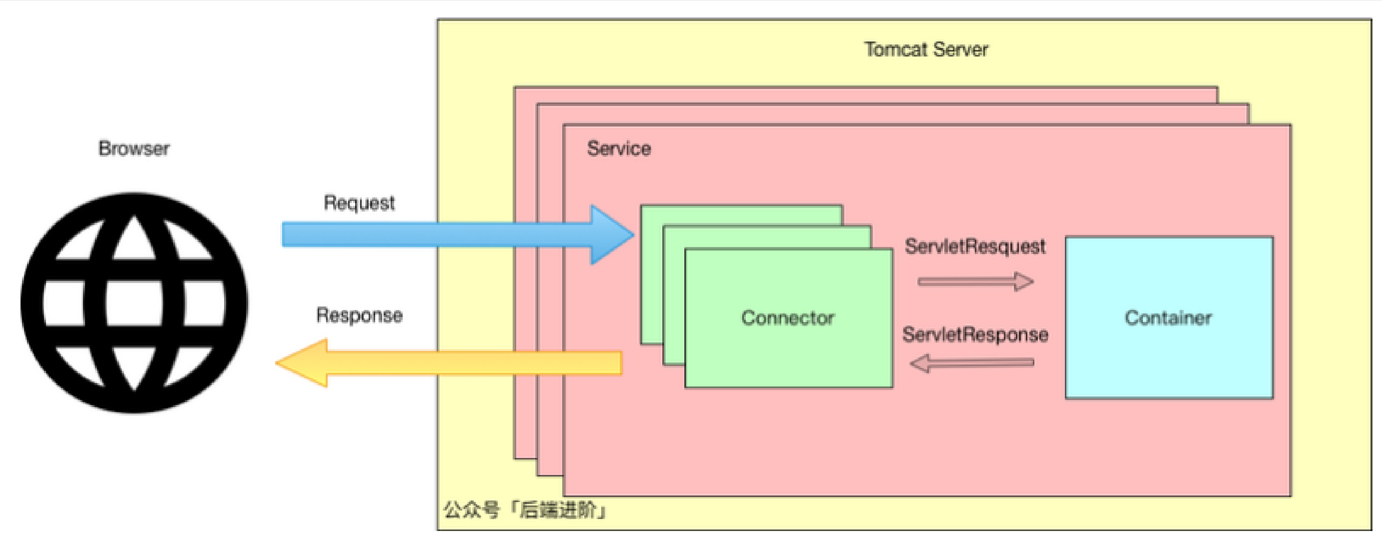
# Application Server Popularity

Tomcat is free and lightweight. It offers basic functionalities needed by many applications




https://www.jrebel.com/resources/java-developer-productivity-report-2024

# Architecture of Tomcat




https://developpaper.com/talking-about-tomcats-architecture-design/

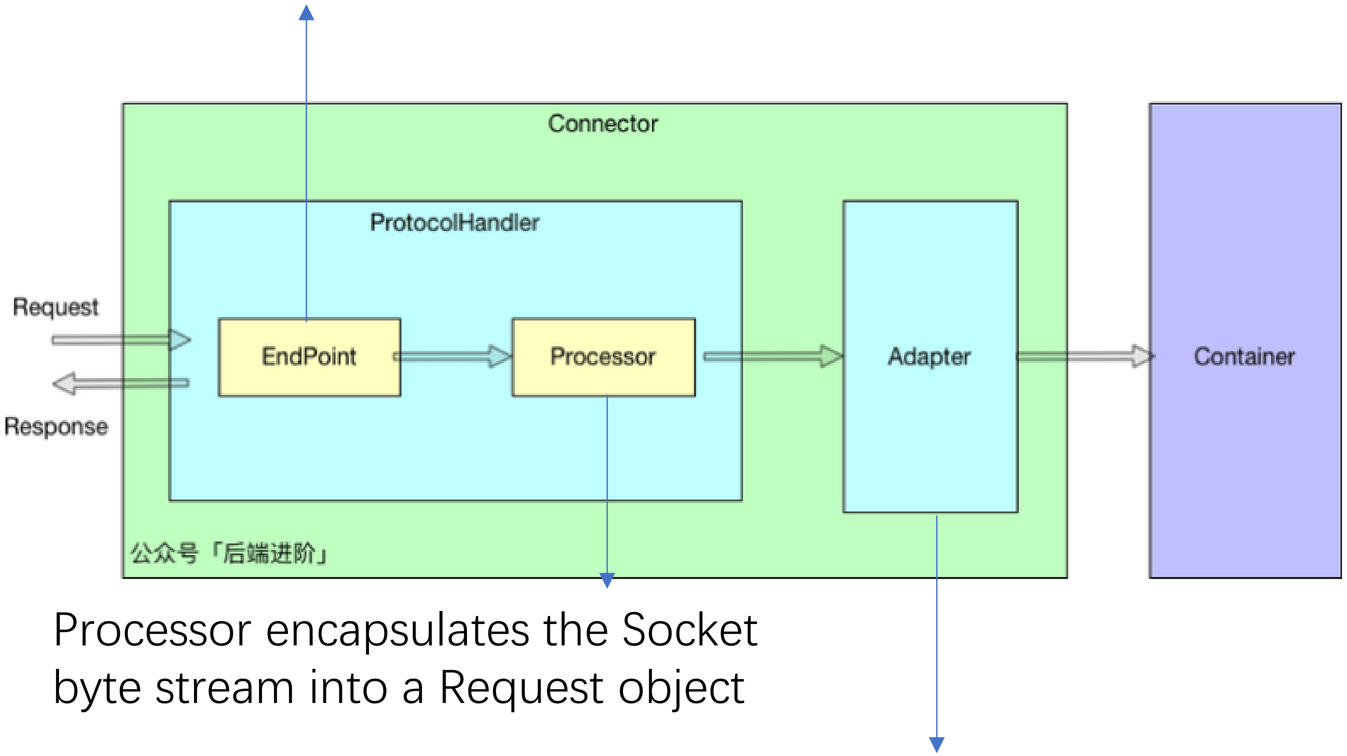
- Tomcat represents a server
- A server can provide multiple services (multiple ports)
- A service can contain multiple connectors
  - Connector handles network connection
  - Multiple connectors support different network protocols
- One service contains only one container/engine, which handles internal Servlets
- Connectors communicate with the container through ServletRequest and ServletResponse objects.

# Connector

- Connector is responsible for encapsulating all kinds of network protocols, shielding the details of network connection and IO processing, and passing the processed Request object to container
- Tomcat encapsulates the details of processing requests to the ProtocolHandler interface

https://developpaper.com/talking-about-tomcats-architecture-design/

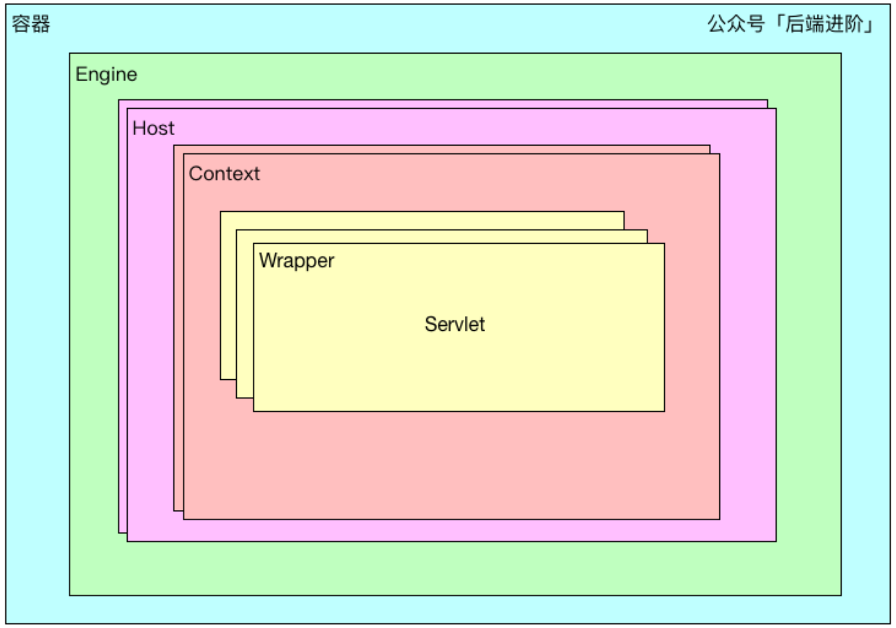
Endpoint components handle underlying Socket network connections




Processor encapsulates the Socket byte stream into a Request object

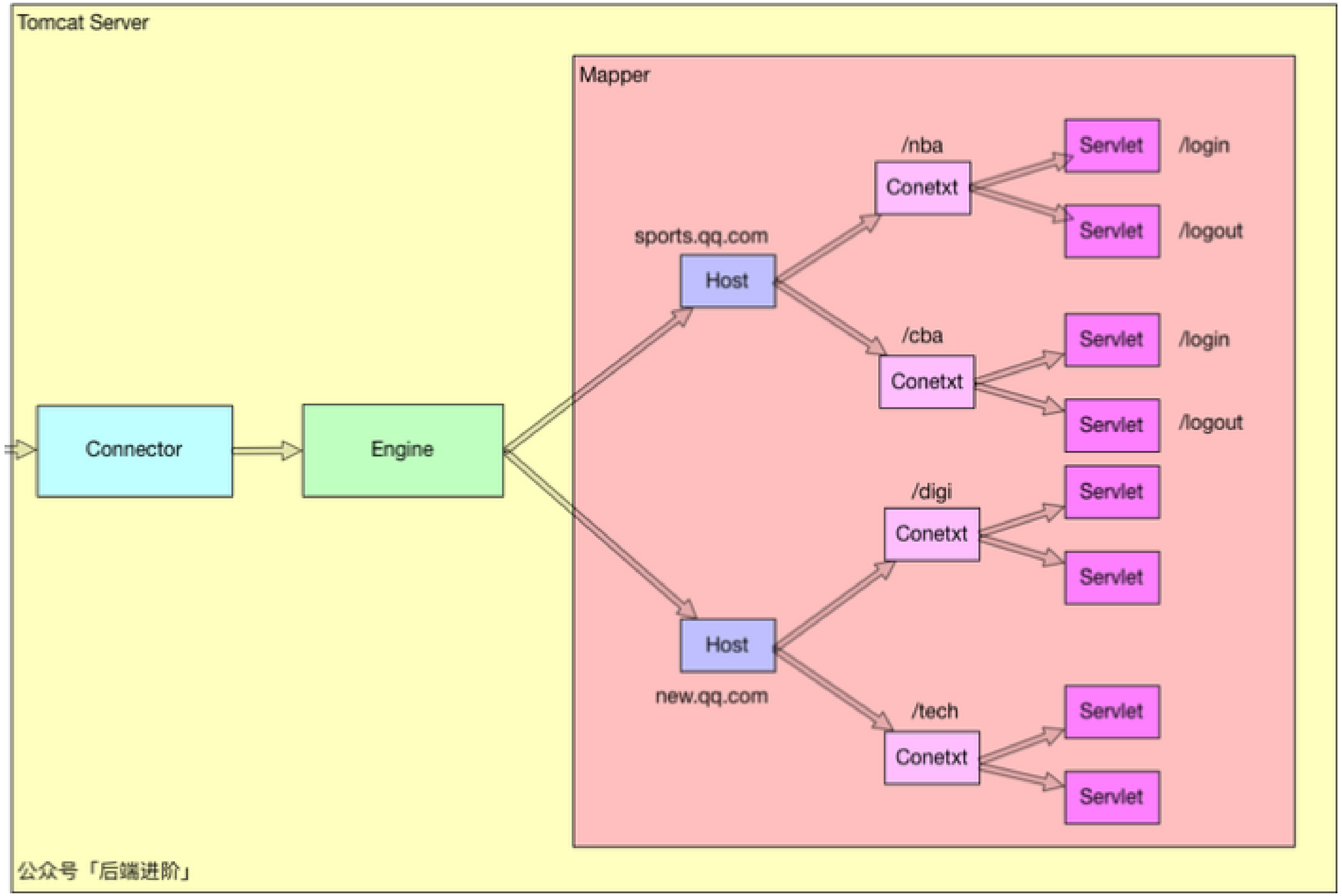
Adapter adapts the Request object to the Request object that the container can recognize

# Container

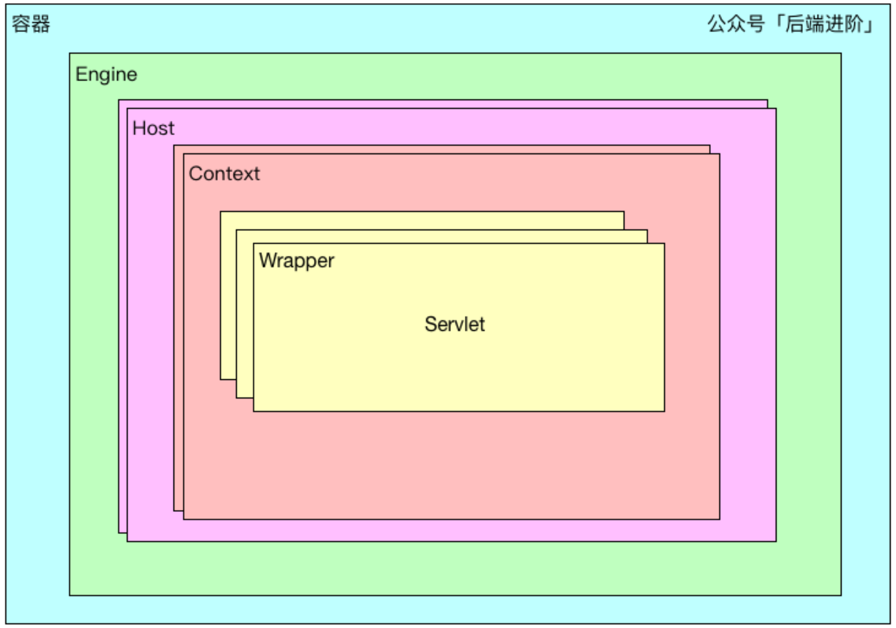



Engine

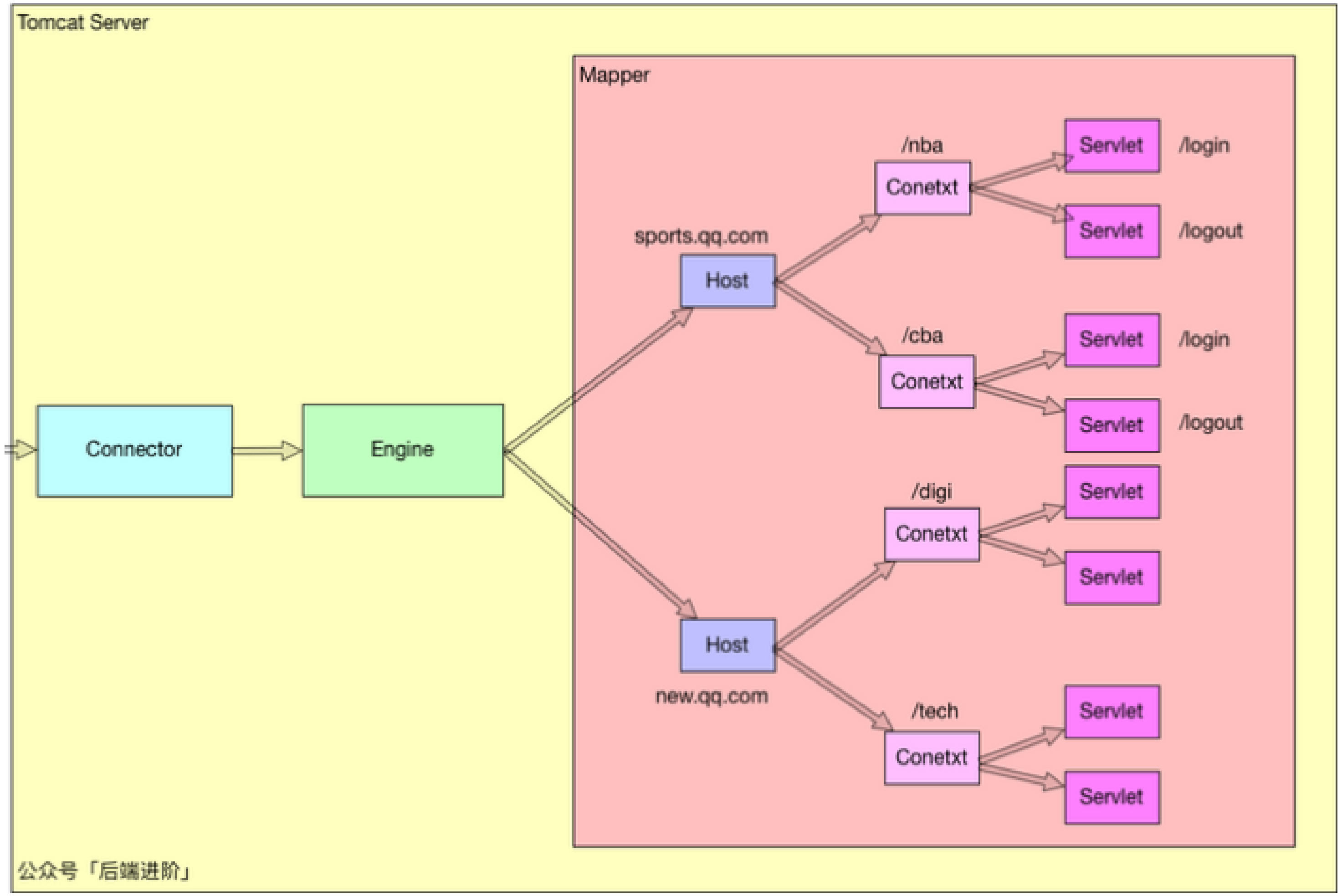
- Engine represents the entire request processing machinery associated with a particular Service.
- All requests of the connector are handed over to the engine, then to the corresponding virtual host



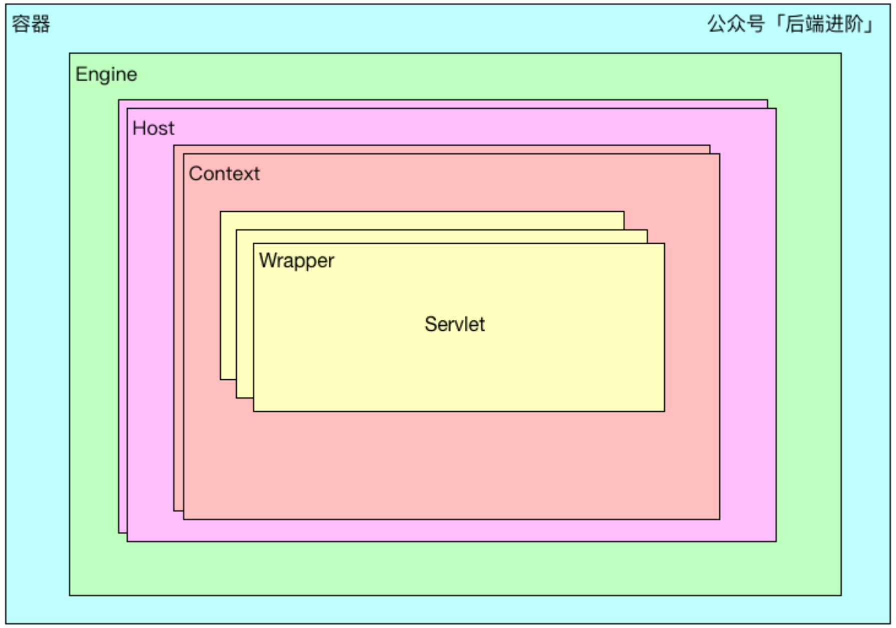
# Container



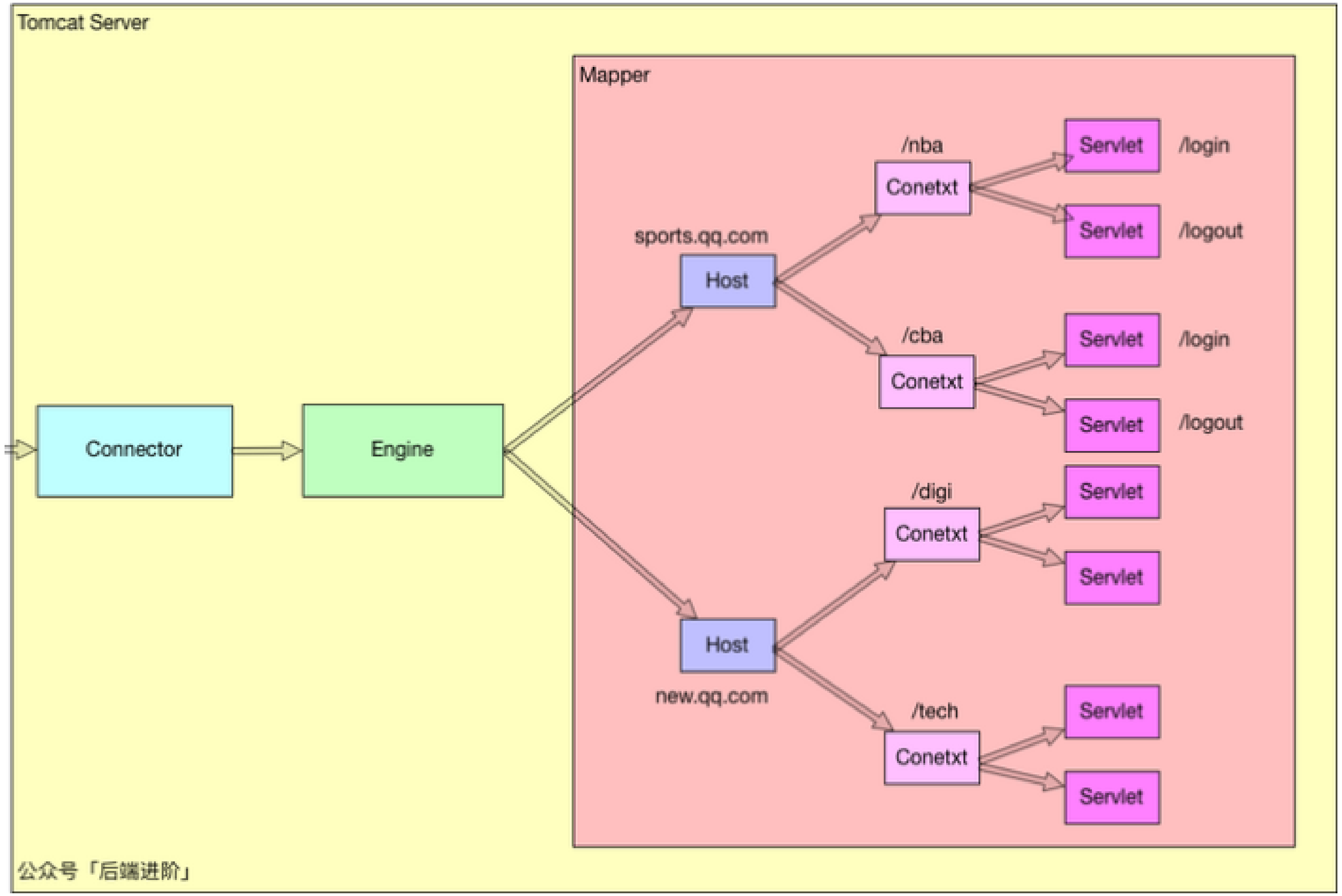

Host
- A virtual host.
- An engine can have more than one virtual host.
- Each host has its own domain name.



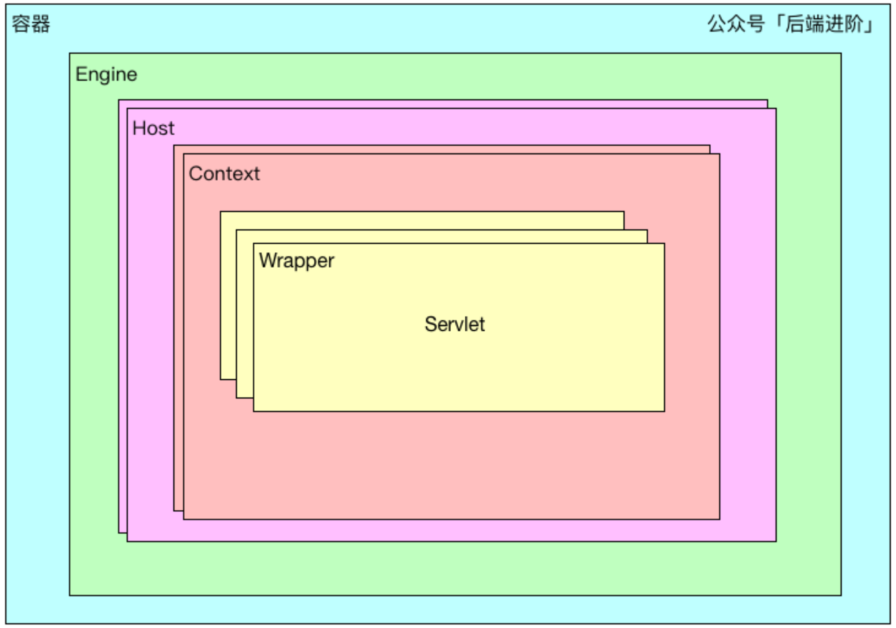
# Container

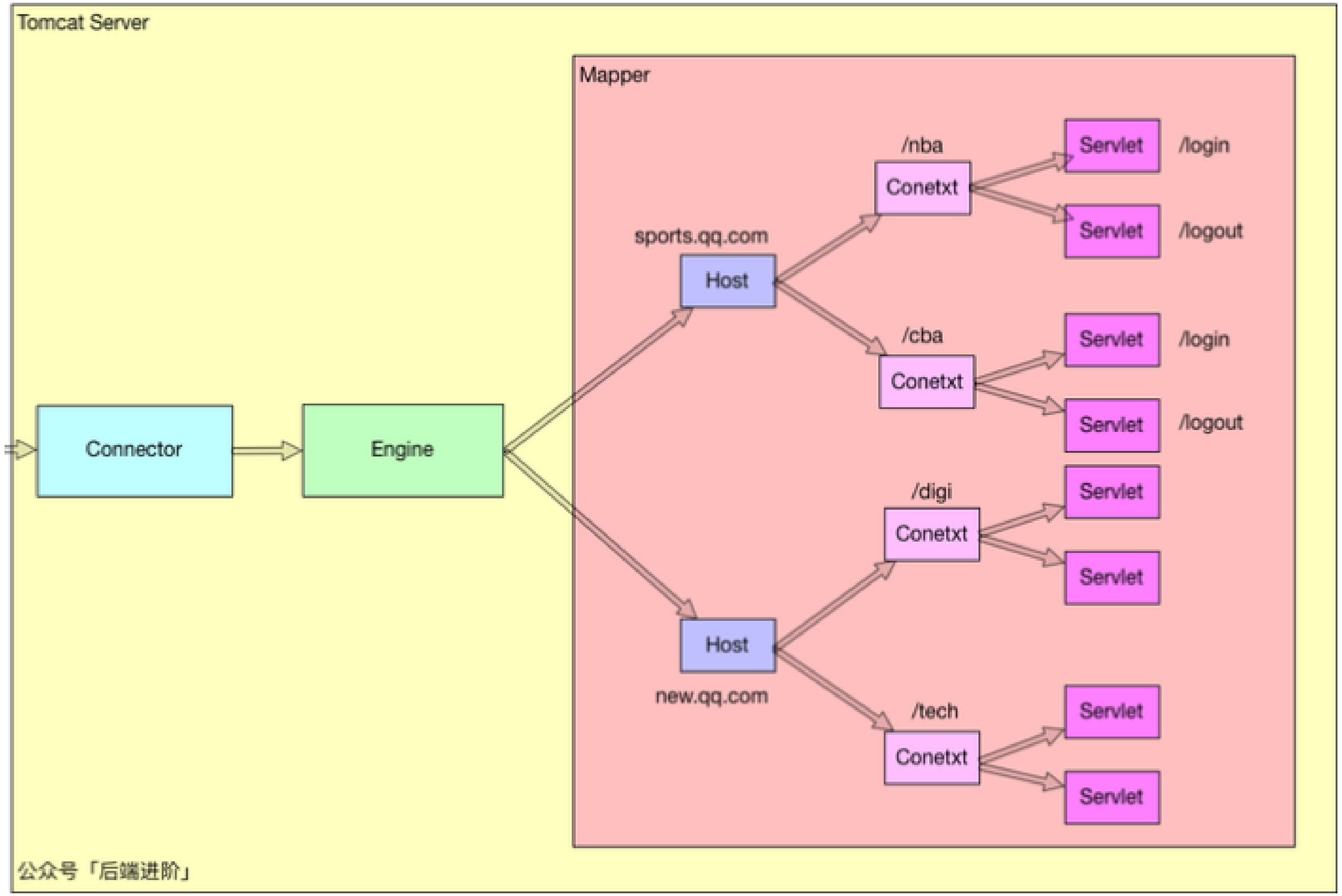



Context
- Represents an application
- A virtual host can have multiple applications
- Each application can configure multiple servlets..



# Container

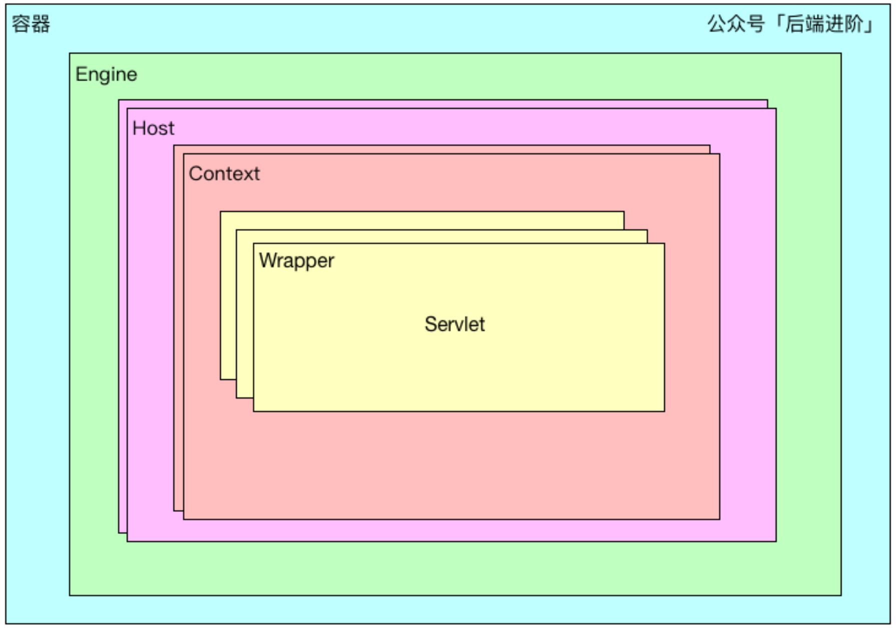


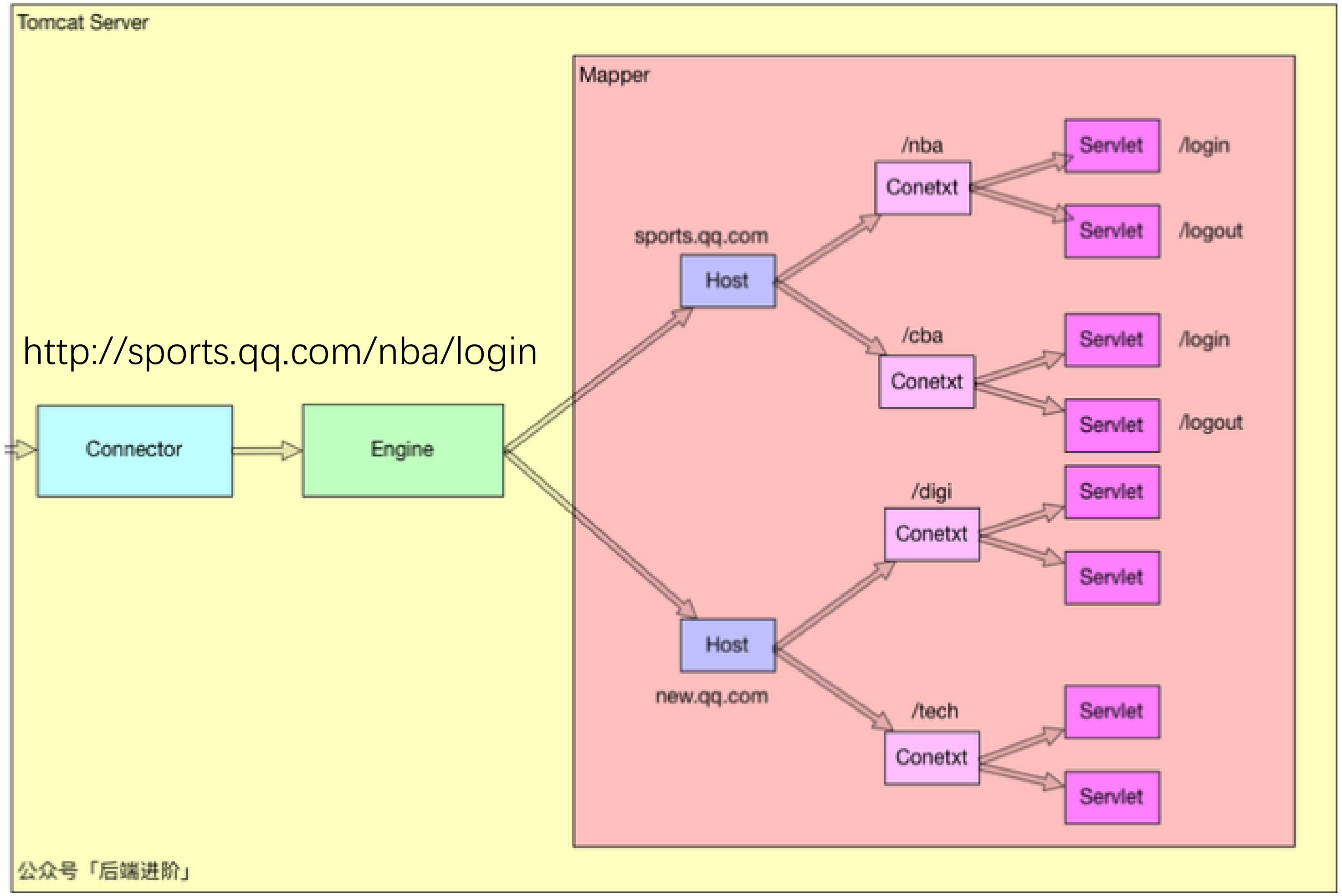




Wrapper

- Represents an individual servlets

# Container

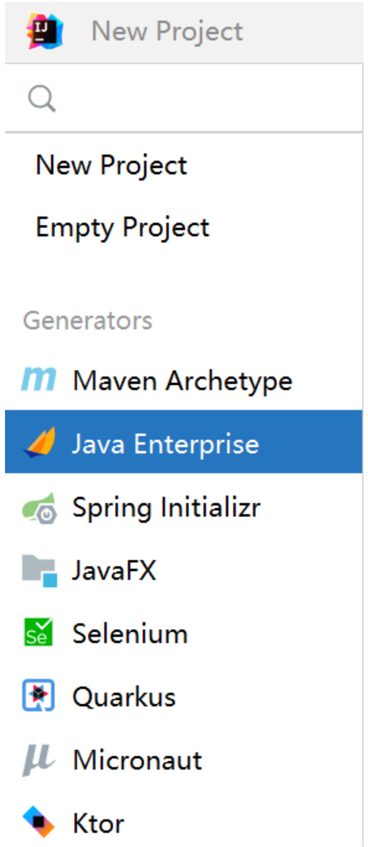


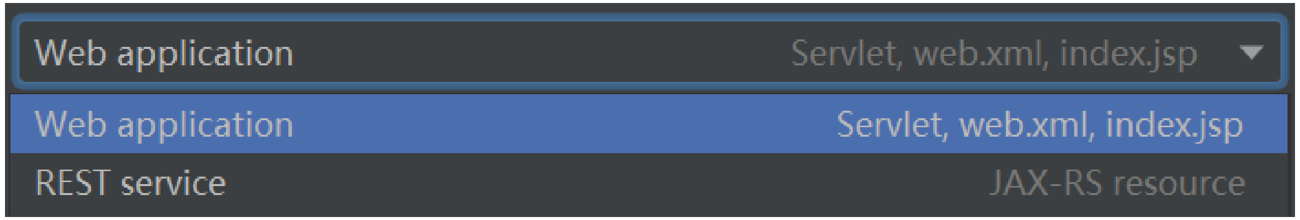



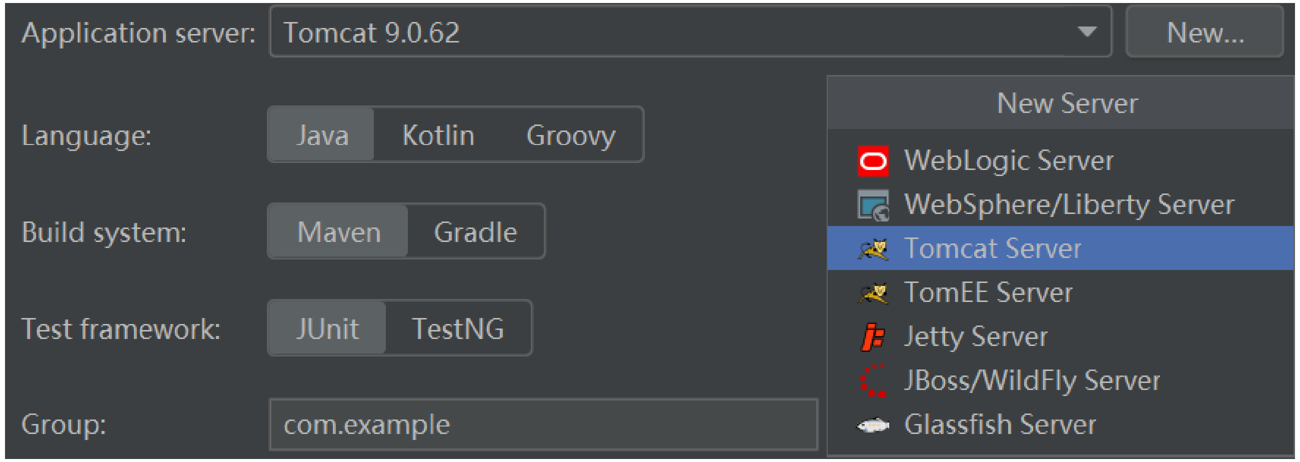

Mapper

- Locate the host and the context given the URL
- Locate the servlet using web.xml

# Working with Tomcat in IntelliJ IDEA

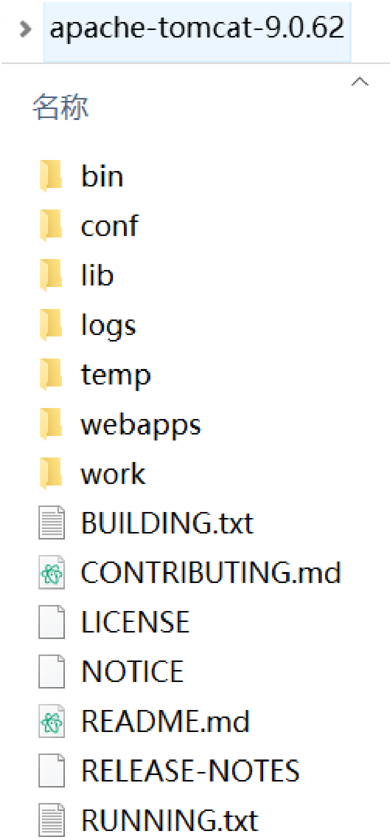


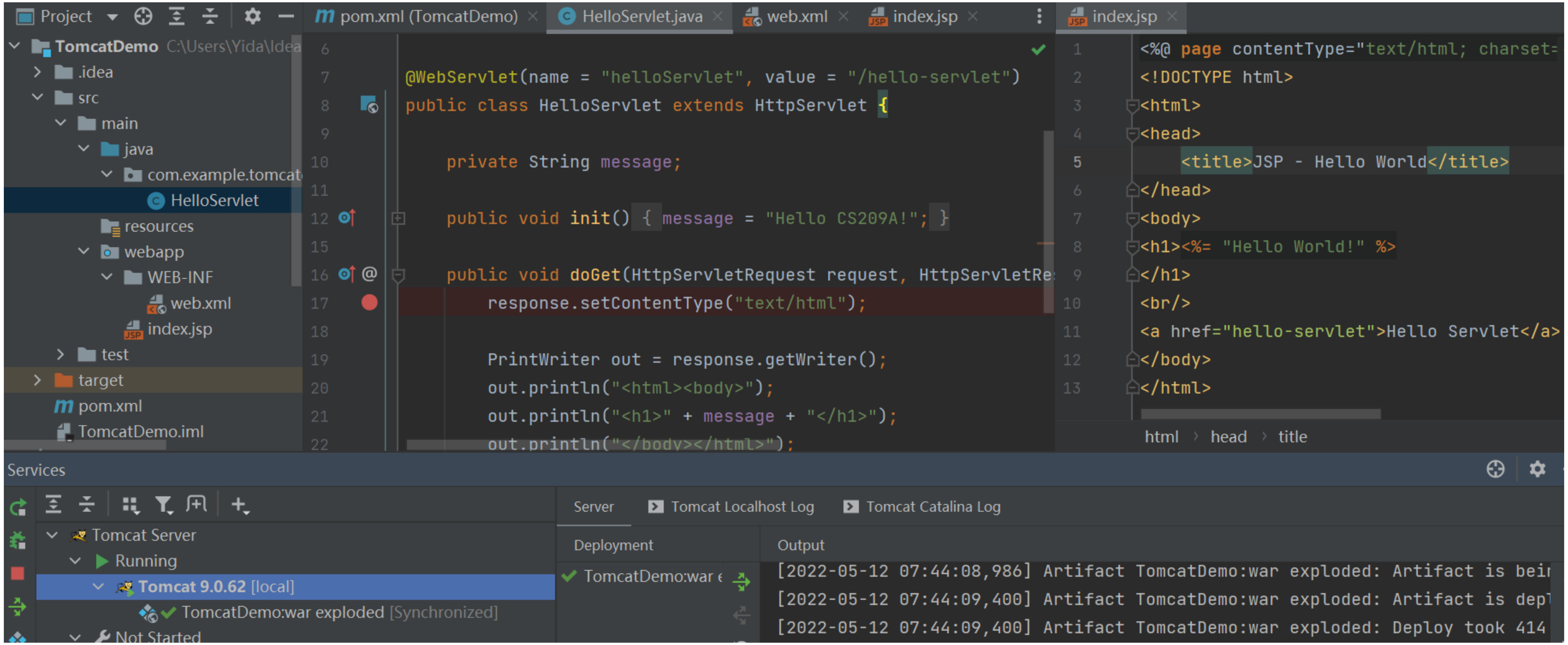

Download tomcat (.zip) and install (unzip) it

Then specify the installation path in IDEA. Done!

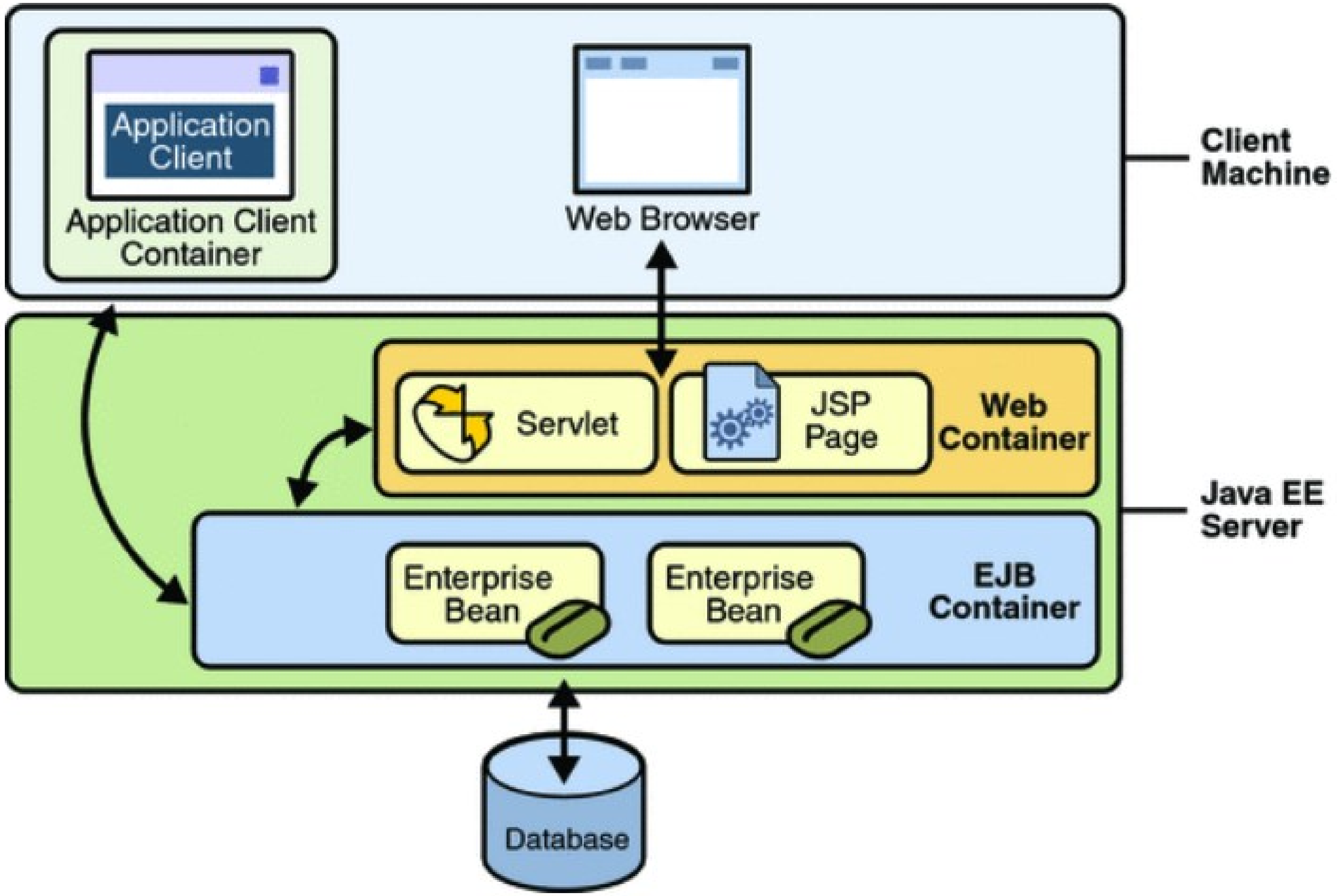


# To put it altogether……

localhost:8080/TomcatDemo/hello-servlet



# What we have learned so far ……



# Lecture 11

- Web Development Overview
- Java EE
- Servlet & Containers
- JDBC & JPA

# Recall Data Persistence (数据持久化)

- Objects created in Java programs live in memory; they are removed by the garbage collector once they are not used anymore
- What if we want to persist the objects?

# File Systems vs Database

File system stores unstructured, unrelated data. Better used when:

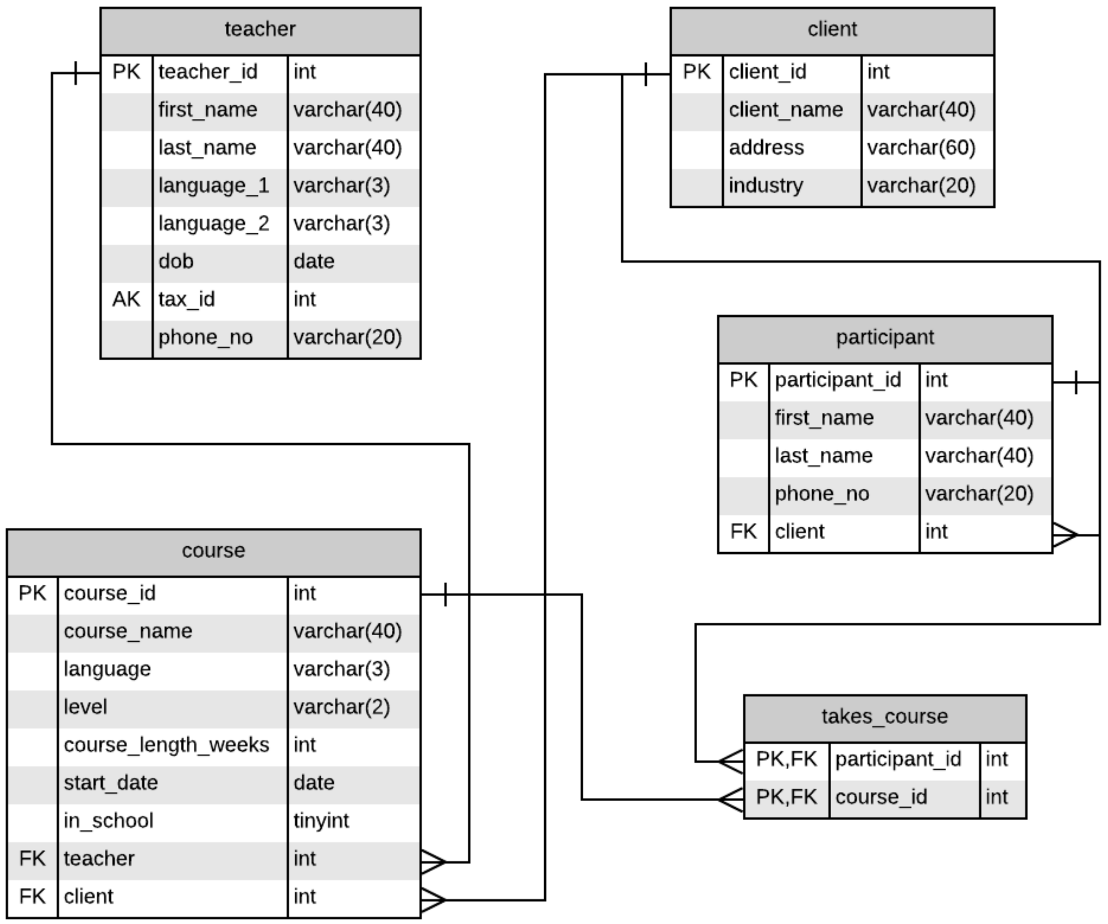
- You like to use version control on your data (a nightmare with dbs)
- You have big chunks of data that grow frequently (e.g., logfiles)
- You want other apps to access your data without API (e.g., text editors)
- You want to store lots of binary content (e.g., pictures, mp3s)

Database stores related, structured data in an efficient manner for insert, update and/or retrieval. Better used when:

- You want to store many rows with the exact same structure
- You need lightning-fast lookup and query processing
- You need to support multiple users and atomic transactions (data safety)

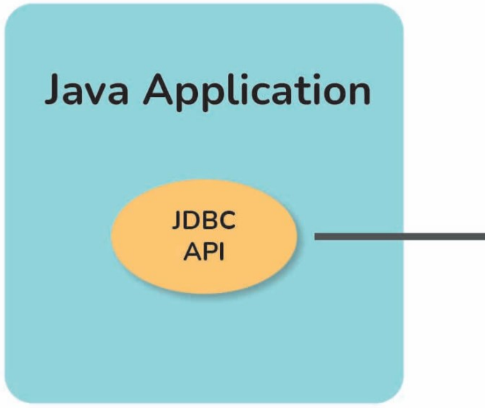
# Relational Database

- A relational database organizes data into rows and columns, which collectively form a table.
- Data is typically structured across multiple tables, which can be joined together via a primary key or a foreign key.
- These unique identifiers demonstrate the different relationships which exist between tables



# JDBC (Java Database Connectivity) API

- To store, organize and retrieve data, most applications use relational databases.
- Java EE applications access relational databases through the JDBC API
- JDBC API are contained in the `java.sql` and `javax.sql` packages

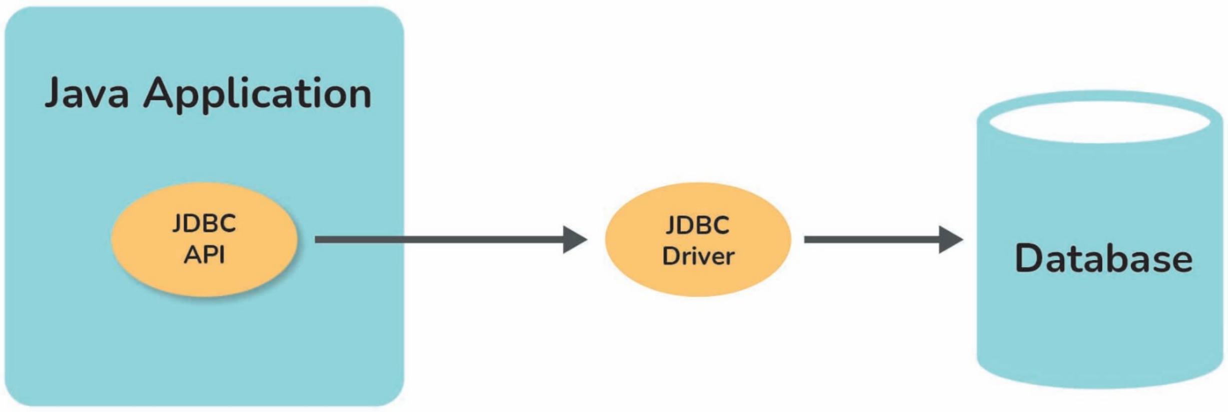



**JDBC API**

- A standard Java API (a set of interfaces and classes) that defines how Java programs should talk to databases.
- Key interfaces: Connection, Statement, ResultSet, etc.
- Provides a uniform, **database-independent way** to send SQL commands, e.g., executeQuery(…)

# JDBC Driver

- A **database-specific implementation** of the JDBC API.
- Provided by database vendors like MySQL, PostgreSQL, etc.
- Translates Java calls into database-specific protocol; Manages low-level communication with the database server
- Install the JDBC driver on the client machine



# Example of Using JDBC https://docs.oracle.com/javase/tutorial/jdbc/TOC.html

```
public Connection getConnection() throws SQLException {

    Connection conn = null;
    Properties connectionProps = new Properties();
    connectionProps.put("user", this.userName);
    connectionProps.put("password", this.password);

    if (this.dbms.equals("mysql")) {
        conn = DriverManager.getConnection(
                   "jdbc:" + this.dbms + "://" +
                   this.serverName +
                   ":" + this.portNumber + "/",
                   connectionProps);
    } else if (this.dbms.equals("derby")) {
        conn = DriverManager.getConnection(
                   "jdbc:" + this.dbms + ":" +
                   this.dbName +
                   ";create=true",
                   connectionProps);
    }
    System.out.println("Connected to database");
    return conn;
}
```

```
public static void viewTable(Connection con) throws SQLException {
  String query = "select COF_NAME, SUP_ID, PRICE, SALES, TOTAL from COFFEES";
  try (Statement stmt = con.createStatement()) {
    ResultSet rs = stmt.executeQuery(query);
    while (rs.next()) {
      String coffeeName = rs.getString("COF_NAME");
      int supplierID = rs.getInt("SUP_ID");
      float price = rs.getFloat("PRICE");
      int sales = rs.getInt("SALES");
      int total = rs.getInt("TOTAL");
      System.out.println(coffeeName + ", " + supplierID + ", " + price +
                         ", " + sales + ", " + total);
    }
  } catch (SQLException e) {
    JDBCTutorialUtilities.printSQLException(e);
  }
}
```

**Separating API and Driver: we can easily switch to another DB without affecting the client code**

# Example of Using JDBC

https://docs.oracle.com/javase/tutorial/jdbc/TOC.html

```
public Connection getConnection() throws SQLException {

    Connection conn = null;
    Properties connectionProps = new Properties();
    connectionProps.put("user", this.userName);
    connectionProps.put("password", this.password);

    if (this.dbms.equals("mysql")) {
        conn = DriverManager.getConnection(
                   "jdbc:" + this.dbms + "://" +
                   this.serverName +
                   ":" + this.portNumber + "/",
                   connectionProps);
    } else if (this.dbms.equals("derby")) {
        conn = DriverManager.getConnection(
                   "jdbc:" + this.dbms + ":" +
                   this.dbName +
                   ";create=true",
                   connectionProps);
    }
    System.out.println("Connected to database");
    return conn;
}
```

```
public static void viewTable(Connection con) throws SQLException {
  String query = "select COF_NAME, SUP_ID, PRICE, SALES, TOTAL from COFFEES";
  try (Statement stmt = con.createStatement()) {
    ResultSet rs = stmt.executeQuery(query);
    while (rs.next()) {
      String coffeeName = rs.getString("COF_NAME");
      int supplierID = rs.getInt("SUP_ID");
      float price = rs.getFloat("PRICE");
      int sales = rs.getInt("SALES");
      int total = rs.getInt("TOTAL");
      System.out.println(coffeeName + ", " + supplierID + ", " + price +
                         ", " + sales + ", " + total);
    }
  } catch (SQLException e) {
    JDBCTutorialUtilities.printSQLException(e);
  }
}
```

**One needs to write SQL queries and manually map between Java object's data and relational DB, which can be complicated**

# Mismatch between Objects and Tables

## Technical difficulties of matching the relational model (DB) and the object model (Java)

| | |
|---|---|
| **Granularity** | The object model has various levels of granularity but a database table has only two, tables and columns, for example you could have two classes Person and Address but only one table that contains both this information. |
| **Inheritance** | objects have the ability to inherit but database tables do not. |
| **Identity** | Databases use a primary key to identify a row but Java uses both object identity (==) and equality (equals) |
| **Associations** | In java you use references to associate objects and they can be bi-directional but in databases we use a foreign key which are not directional. |
| **Data Navigation** | In Java you use the object graph to walk the associations, for example a Person object may contain references to an Address Object which in turn has references to a PostCode object, in order to get to the PostCode object you have walk both Person and Address objects. Databases use SQL joins, which joins tables together to retrieve data. |

http://www.datadisk.co.uk/html_docs/java_persistence/persistence_1.html

# Object-Relational Mapping (ORM)

- ORM techniques/libraries let us query and manipulate data from a database using an object-oriented paradigm
- We don't use SQL anymore; we interact directly with Java object

```
List<Student> list = studentRepository.findAll();
```

With ORM libraries

```
List<Student> students = new ArrayList<>();

String sql = "SELECT id, name FROM student";

try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

   while (rs.next()) {
      Student s = new Student();
      s.setId(rs.getInt("id"));
      s.setName(rs.getString("name"));
      students.add(s);
   }

} catch (SQLException e) {
   e.printStackTrace();
}
```

With plain JDBC

# Java Persistence API (JPA)

- JPA is the Java EE standard specification for ORM.
- Reference implementation
  - Eclipselink (used in GlassFish)
- Other implementations
  - Hibernate
  - Apache OpenJPA

# An Analogy of JPA vs Hibernate

```java
public interface JPA {

        public void insert(Object obj);

        public void update(Object obj);

        public void delete(Object obj);

        public Object select();

}
```

http://tothought.com/post/2

```java
public class Hibernate implements JPA {

    public void insert(Object obj) {
   //Persistence code
    }

    public void update(Object obj) {
       //Persistence code

    }

    public void delete(Object obj) {
      //Persistence code

    }
```

# An Analogy of JPA vs Hibernate

```
public class MyApplication {

        public static JPA jpa = new Hibernate();

        public static void main(String[] args) {
                Object object = new Object();
                jpa.insert(object);  //writes to DB
        }
}
```

We could switch to other JPA implementations easily

http://tothought.com/post/2

# Hibernate Architecture

- Persistence logic
  - Hibernate Configuration: e.g., how to connect to the database
    Hibernate mapping: e.g., how to map a class to a table
- Entity classes: normal Java classes
- Client manipulate data objects using normal OO syntax (e.g., book.setName(“xxx”)), which interact with the Hibernate framework underneath
- Hibernate in turn interacts with JDBC, JNDI, JTA to connect to the database to perform that persistence logic, with the help of JDBC driver

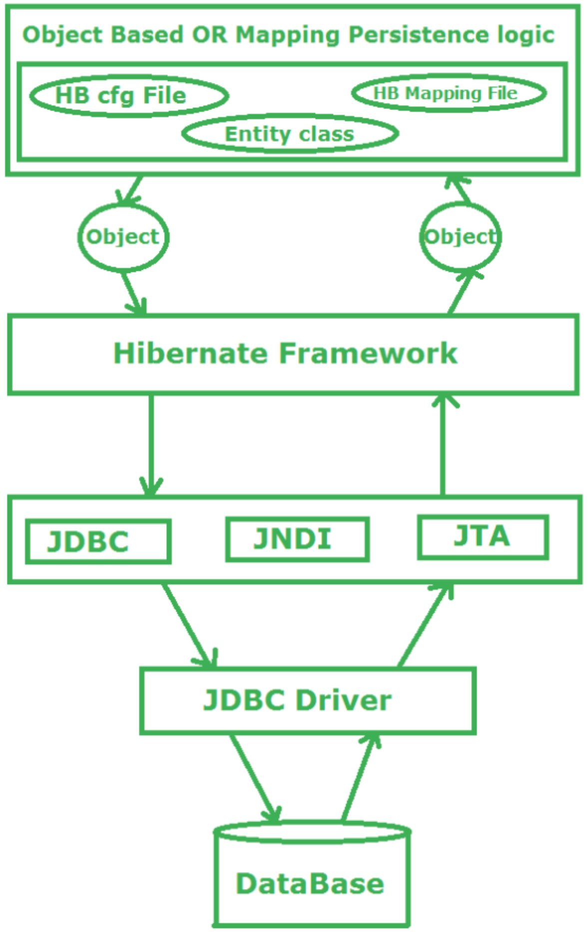
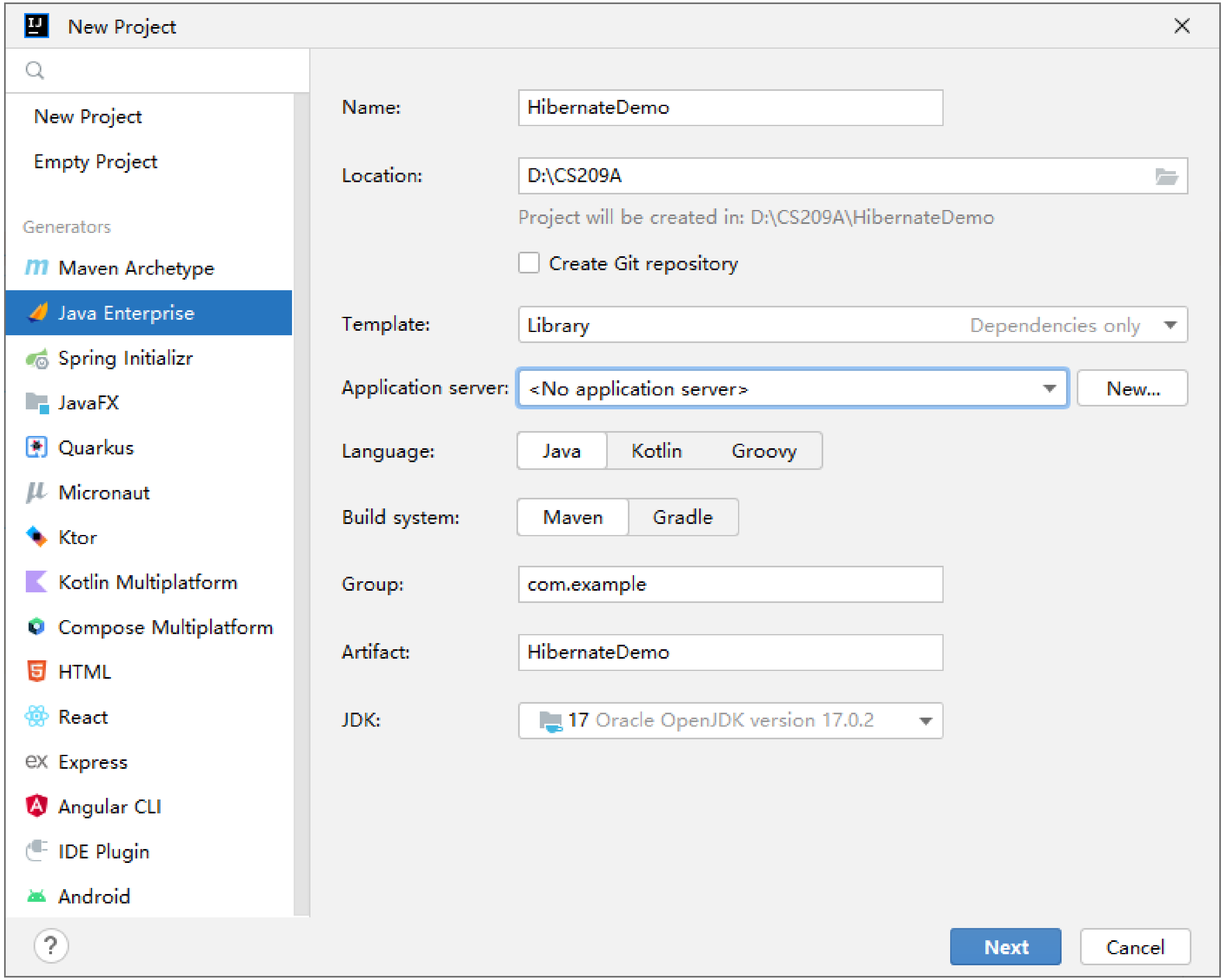
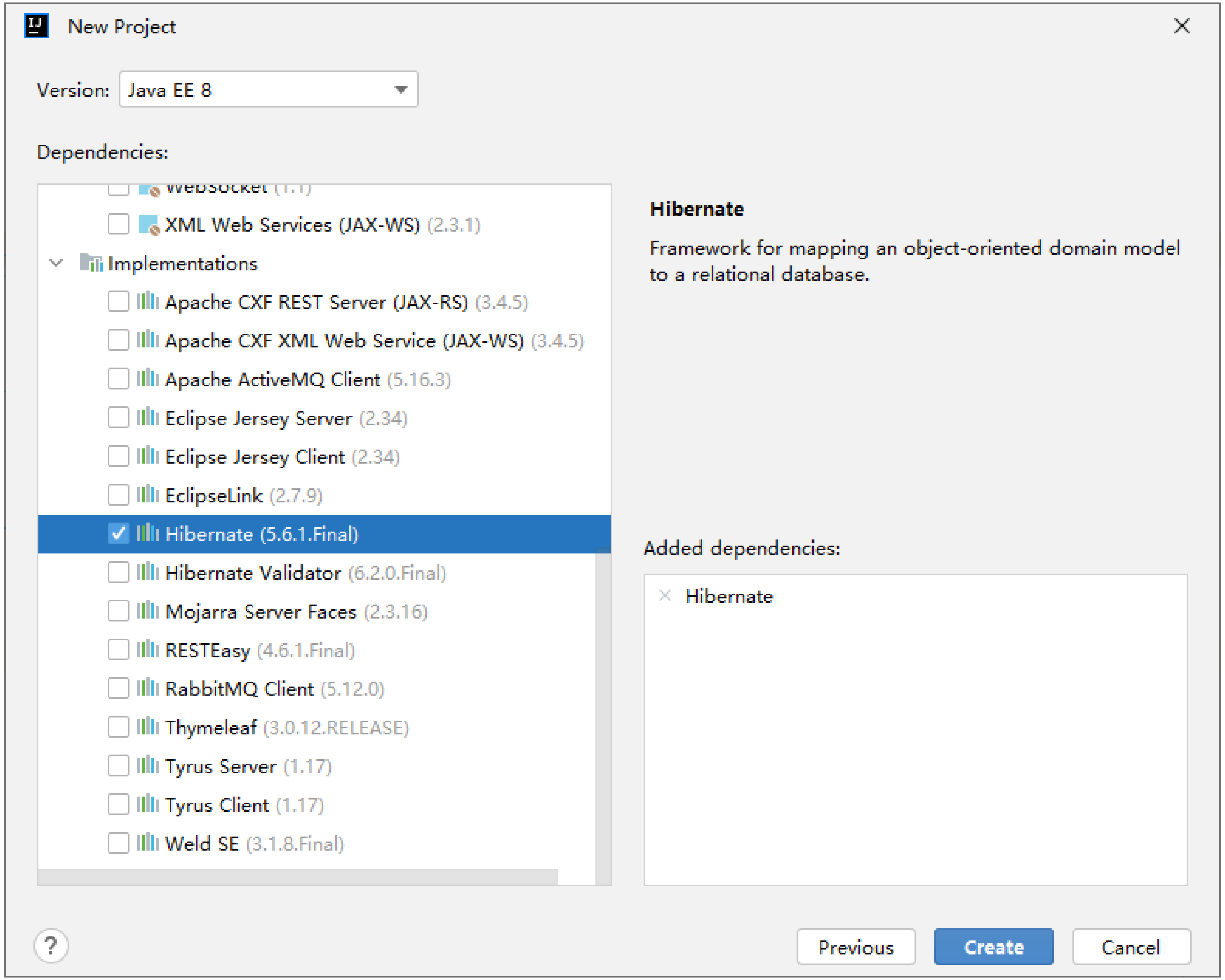



**Fig: Working Flow of Hibernate framework to save/retrieve the data from the database in form of Object**

https://www.geeksforgeeks.org/hibernate-architecture/

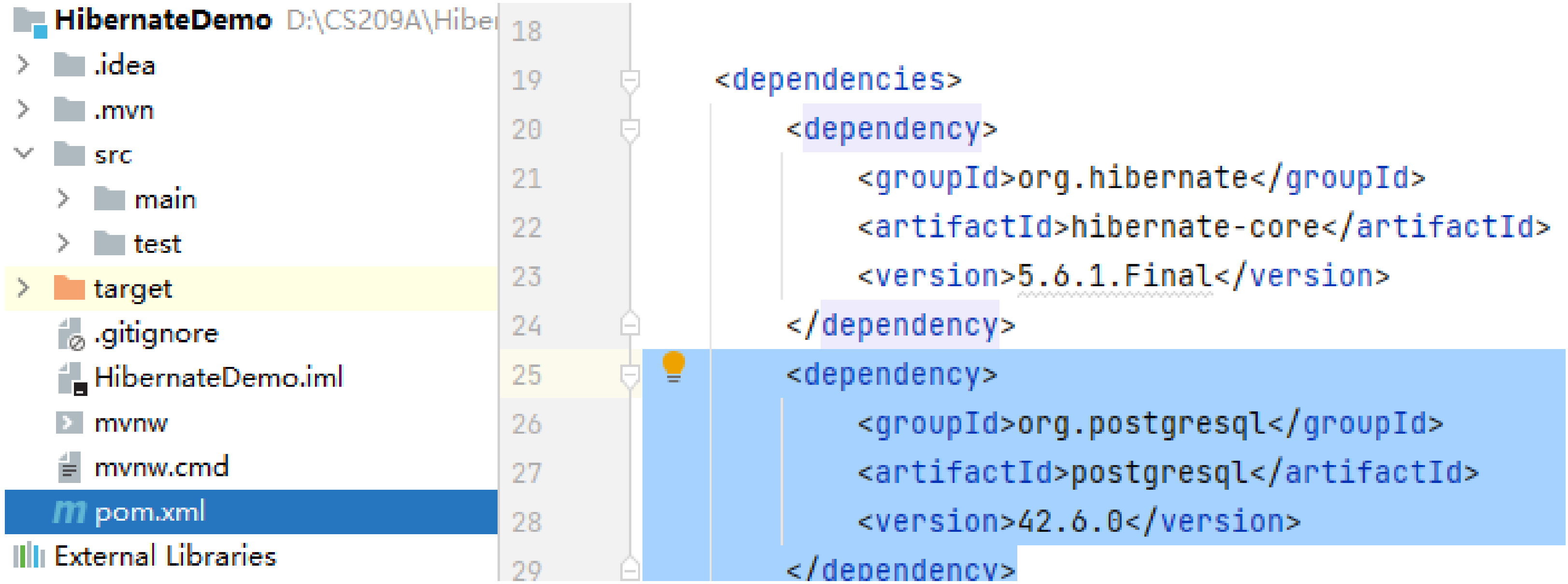



# Working with Hibernate in IntelliJ IDEA



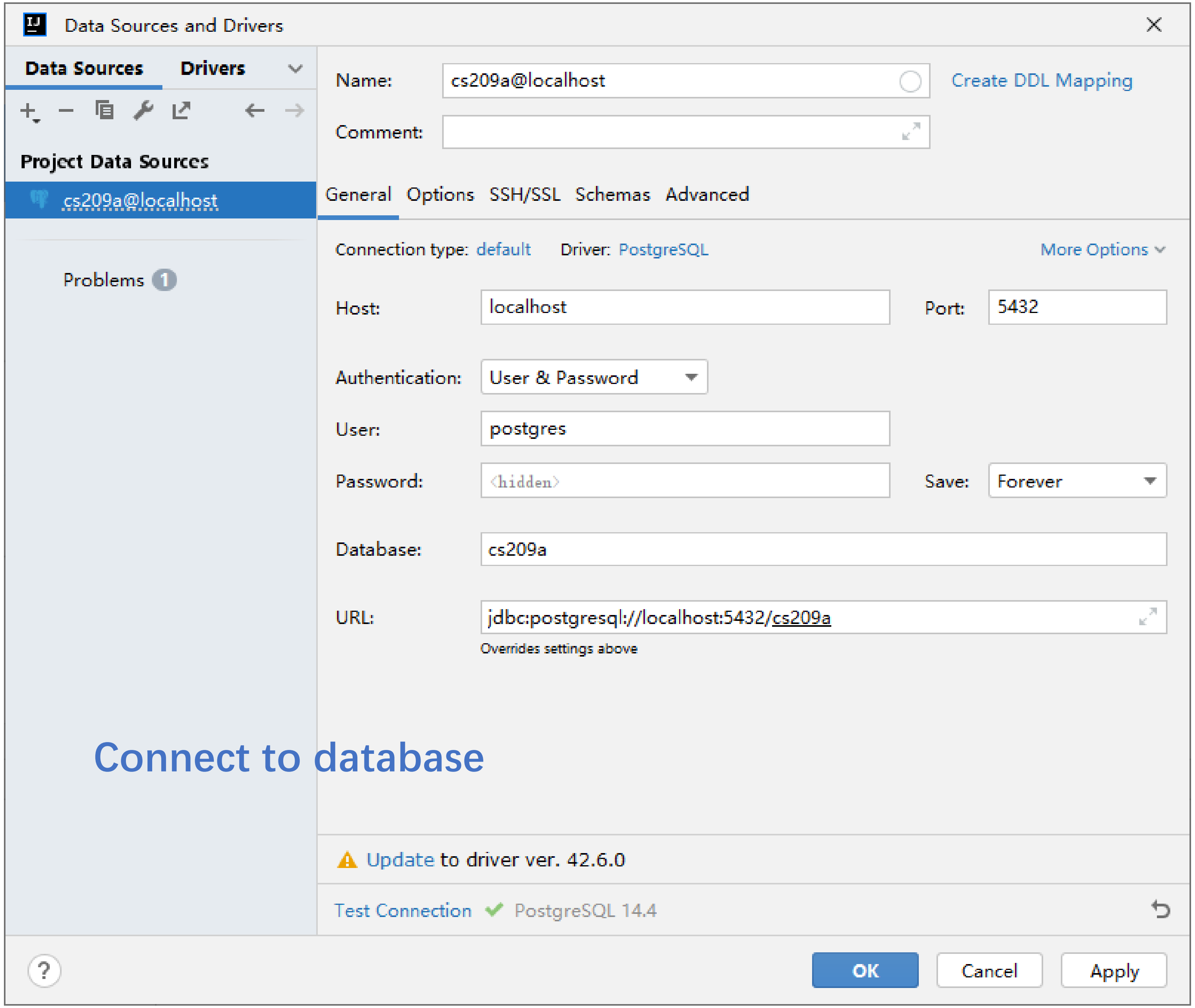

# Working with Hibernate in IntelliJ IDEA

# Working with Hibernate in IntelliJ IDEA

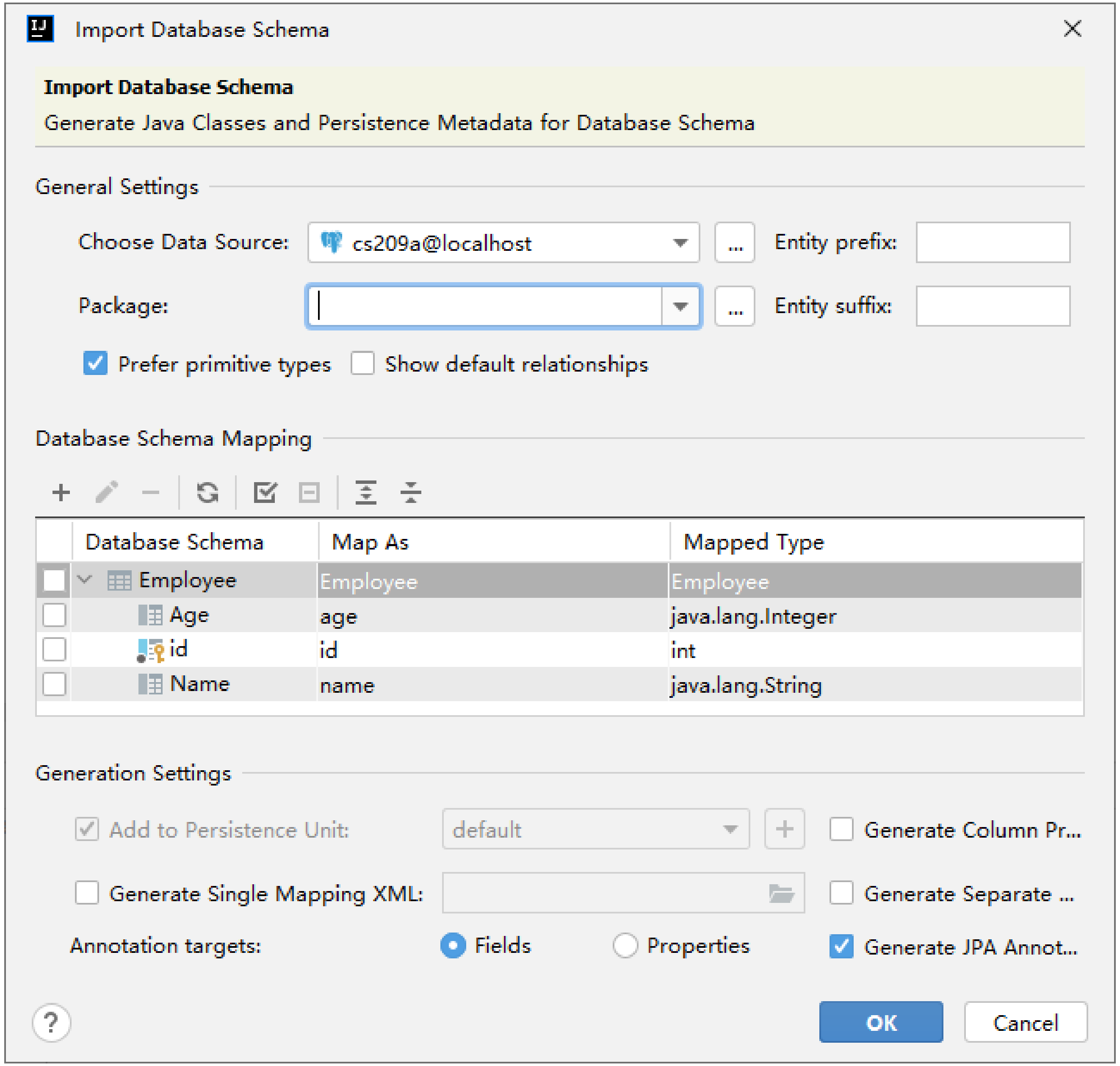



- Hibernate dependency
- Driver dependency (manually added)

# Working with Hibernate in IntelliJ IDEA

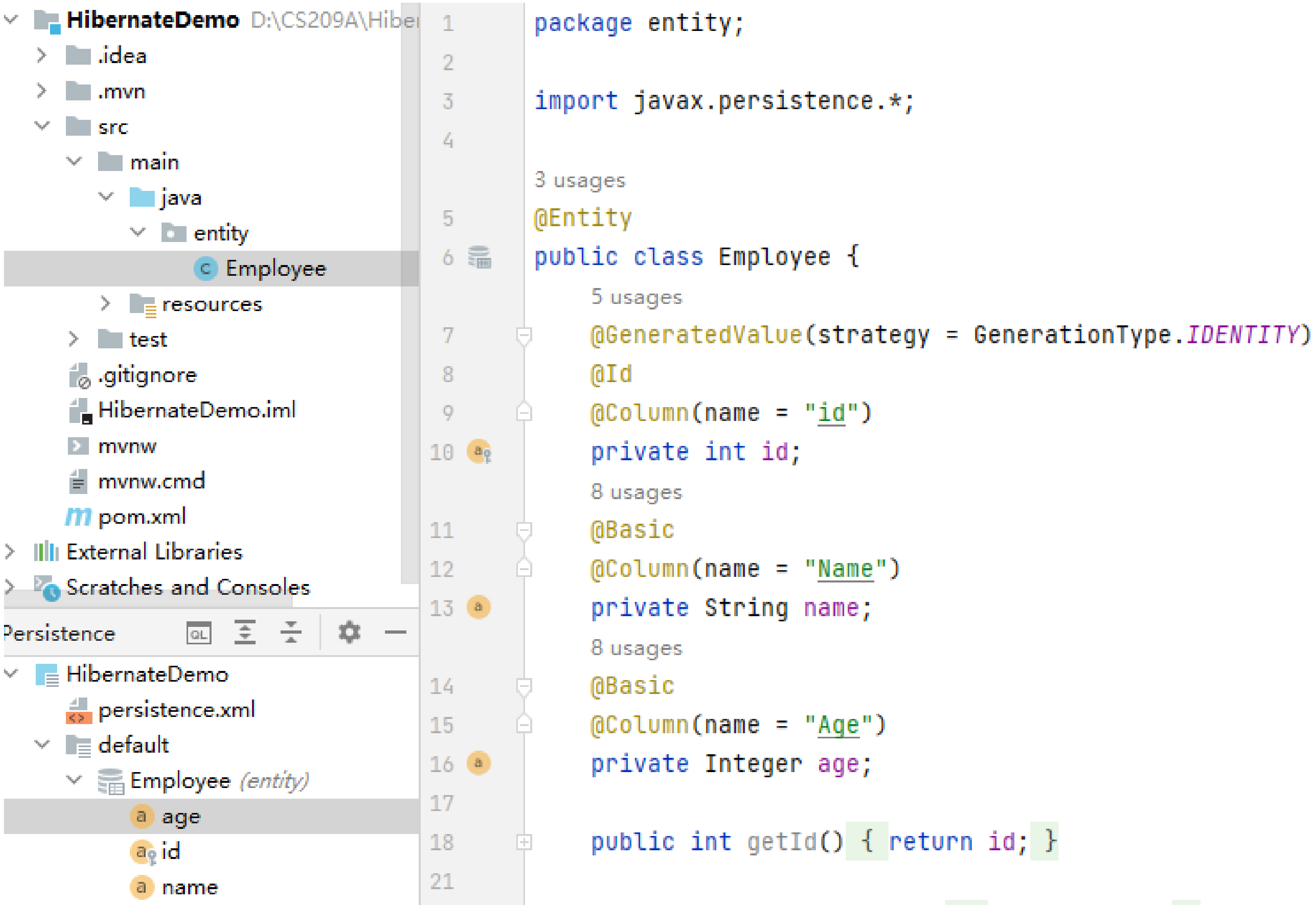



Connect to database



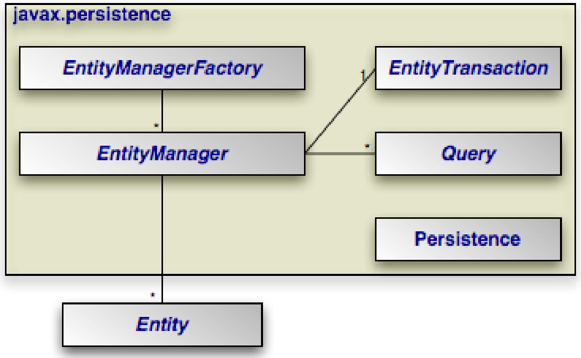

# Working with Hibernate in IntelliJ IDEA

**Configuring mappings between entity classes and db table & columns**



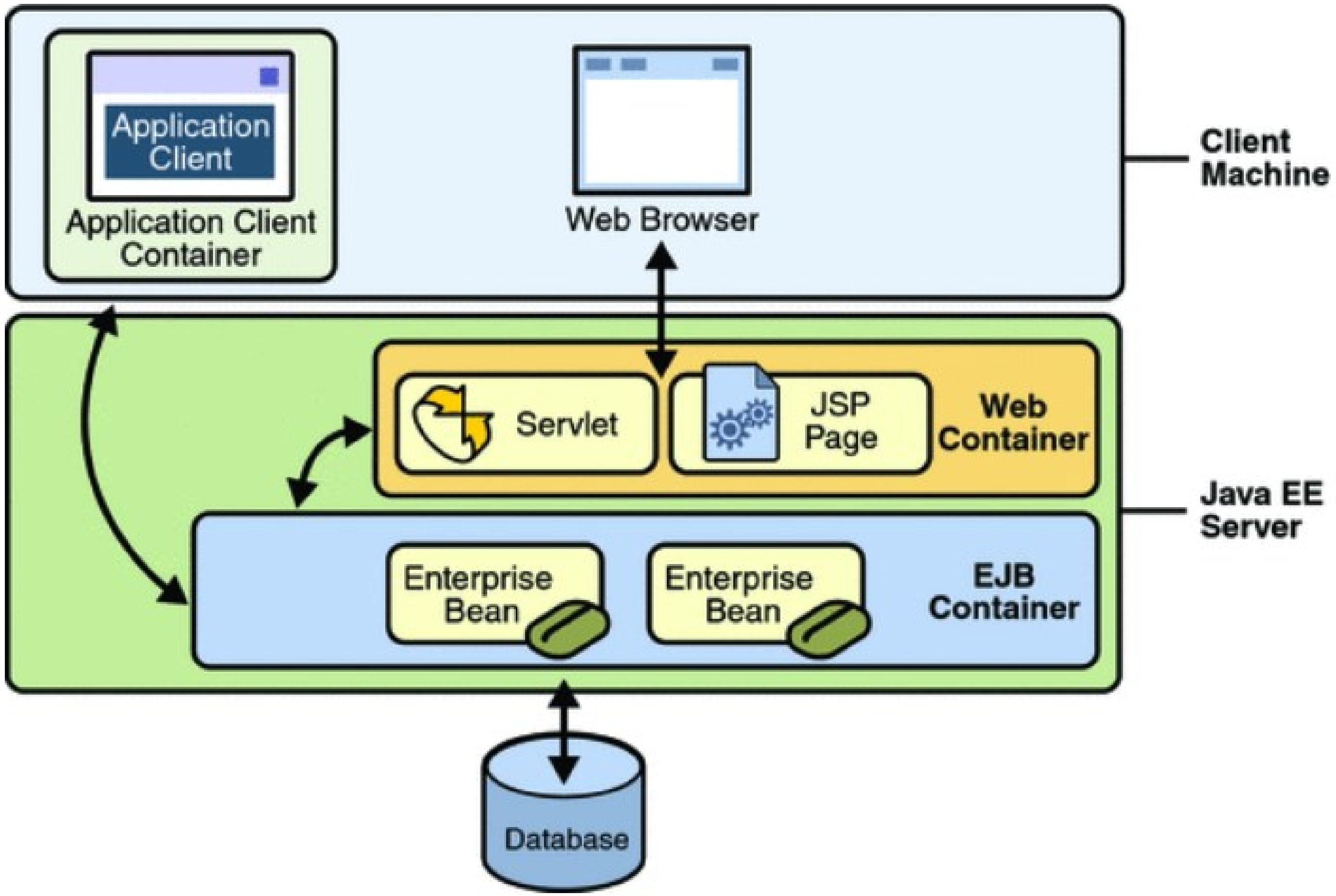

# Working with Hibernate in IntelliJ IDEA

```java
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory(
        EntityManager entityManager = entityManagerFactory.createEntityManager();
        EntityTransaction transaction = entityManager.getTransaction();

        try{
            transaction.begin();

            Employee carl = new Employee();
            carl.setName("Carl");
            carl.setAge(20);

            entityManager.persist(carl);

            transaction.commit();
        }finally {
            if(transaction.isActive()){
                transaction.rollback();
            }
            entityManager.close();
            entityManagerFactory.close();
        }
    }
}
```




# Working with Hibernate in IntelliJ IDEA

# Summary

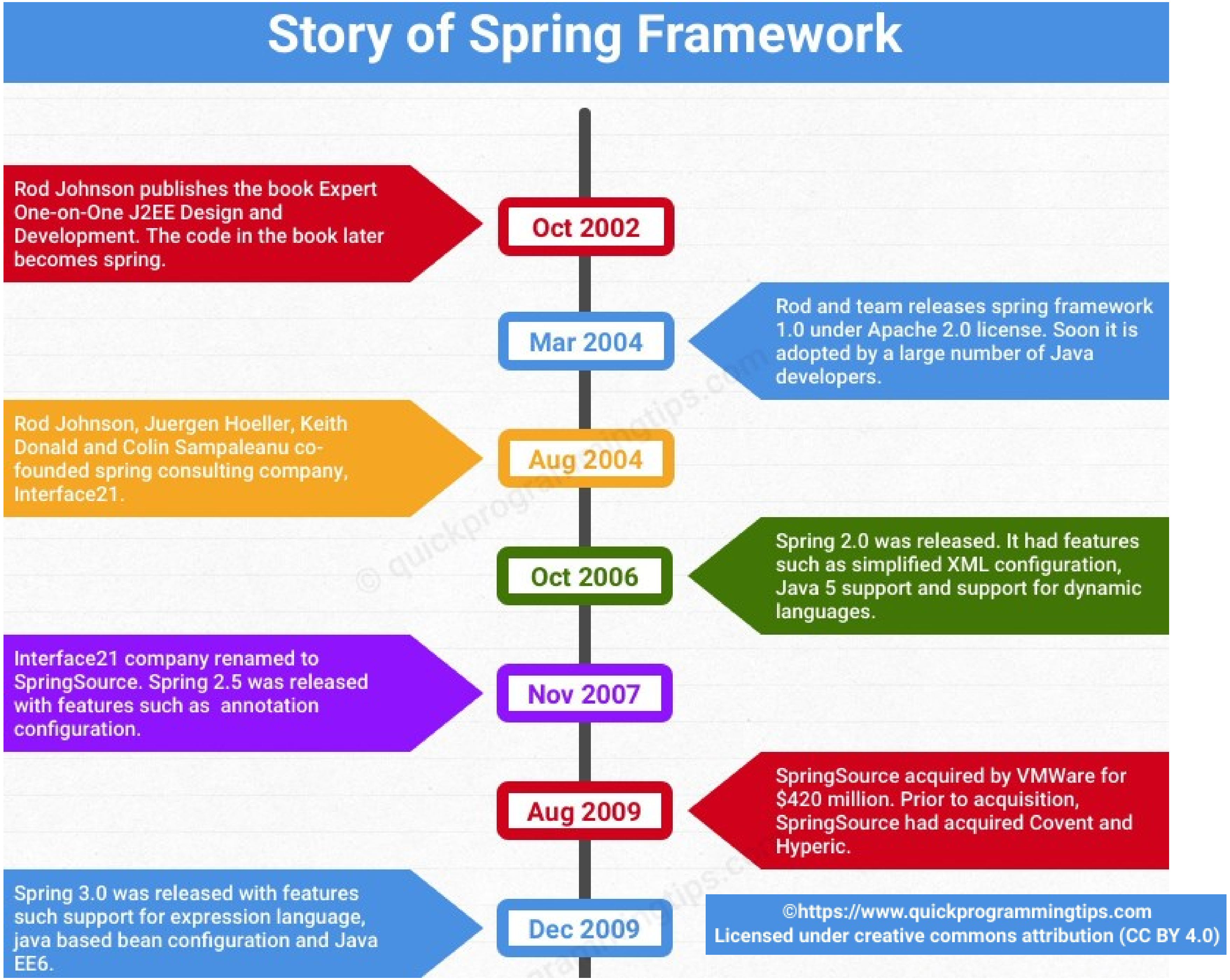


Application Client
Application Client Container
Web Browser
Client Machine
Servlet
JSP Page
Web Container
Java EE Server
Enterprise Bean
Enterprise Bean
EJB Container
Database


Rod Johnson
Rod Johnson is an enterprise Java architect specializing in scalable web applications. He has worked with both Java and J2EE since their release, and he is a member of JSR 154 Expert Group defining the Servlet 2.4 specification.
expert one-on-one
J2EE™ Design and Development
wrox
Updates, source code, and Wrox technical support at www.wrox.com

- This book covered the state of Java enterprise application development at the time and pointed out a number of major deficiencies with Java EE and EJB component framework.
- The book proposed a simpler solution based on POJO and **dependency injection**
- The book shows a high quality, scalable online seat reservation application can be built without using EJB. For building the application, Rod wrote over 30,000 lines of infrastructure code! It included a number of reusable java interfaces and classes such as ApplicationContext and BeanFactory
- The book is an instant hit. Much of the infrastructure code freely provided as part of the book was highly reusable and soon a number of developers started using it in their projects

https://www.quickprogrammingtips.com/spring-boot/history-of-spring-framework-and-spring-boot.html



# Next Lecture

- The Spring Framework
- Spring Boot